



Rapport de stage

Platform Engineering : Création d'un Golden Path

Organisme d'accueil :

INSEE - DSI - DPII

Équipe service KubeApp, KubeSocle

Auteur :

DJIDJELLI Rayane

Maître de stage :

ENEMAN Donatien

Référent pédagogique :

DENEUVILLE Ludovic

24 août 2026

Résumé

Au sein de la Direction du Système d'Information (DSI) de l'INSEE, la modernisation de l'infrastructure (Kubernetes, GitOps, Infrastructure as Code) s'est accompagnée d'une complexité croissante pour les équipes de développement. Une enquête interne menée auprès des développeurs a mis en évidence plusieurs irritants majeurs : dispersion de la documentation, dépendance critique à un nombre restreint d'experts, opacité des pipelines de CI/CD et intégration tardive de la sécurité. Ces frictions ralentissent le *time-to-market*, favorisent l'accumulation de dette technique et limitent la capacité de la DSI à absorber la croissance de son parc applicatif.

Ce mémoire de stage propose de répondre à ces constats à travers le prisme du Platform Engineering, une discipline visant à construire des plateformes internes en libre-service pour réduire la charge cognitive des développeurs et accélérer la livraison de valeur. Le concept central de cette démarche, le Golden Path, désigne un chemin balisé, outillé et documenté permettant à un développeur de passer de l'idée à la production en conservant nativement la conformité aux standards internes.

La mise en œuvre de ce Golden Path a été structurée en quatre étapes complémentaires. La première porte sur l'initialisation des projets, via le Scaffolders de Backstage, qui permet d'instancier en quelques clics un projet conforme aux standards de sécurité et d'architecture de la DSI. La seconde industrialise l'intégration continue en s'appuyant sur le catalogue de composants communautaire To Be Continuous, mutualisant l'effort de maintenance de l'usine logicielle. La troisième s'attache au déploiement continu : après une étude comparative de Backstage, Kratix et Crossplane, une architecture GitOps combinant GitLab CI, ArgoCD et Crossplane a été retenue et validée par un Proof of Concept, offrant une abstraction complète de la complexité Kubernetes. Enfin, la quatrième étape unifie l'ensemble de ces briques au sein d'un portail développeur unique (Internal Developer Platform), également matérialisé par Backstage, à l'issue d'un arbitrage face à plusieurs solutions du marché (GitLab, Port, Harness).

Les travaux menés démontrent qu'il est possible de concilier la liberté et l'autonomie recherchées par les développeurs avec le niveau de gouvernance et de sécurité exigé par une DSI. Les prototypes et preuves de concept réalisés durant ce stage posent les fondations d'une plateforme évolutive, dont le déploiement à l'échelle de l'ensemble des équipes de la DSI constitue la prochaine étape.

Table des matières

Introduction	3
1 Environnement et État des Lieux	4
1.1 Organisation et équipe d'accueil	4
1.2 Existant informatique	4
1.3 Pratiques DevOps	5
2 Problématique et Limites du Modèle Actuel	6
2.1 Analyse de l'enquête interne : Irritants et points de friction	6
2.2 Synthèse des besoins exprimés par les équipes	6
2.3 Conséquences organisationnelles et techniques	7
3 Le Platform Engineering : Une Approche Centrée Produit	8
3.1 Origine et évolution	8
3.2 Les piliers du Platform Engineering	8
3.3 Acteurs et Personnas : À qui s'adresse la plateforme	9
3.4 Le concept de Golden Path (Le "Chemin Pavé")	9
3.5 Comment la plateforme répond-elle à nos besoins	10
4 Étape 1 : Initialisation et Création de Projets	11
4.1 Expression du besoin	11
4.2 Étude comparative des solutions	11
4.3 Solution retenue et architecture	12
4.4 Mise en œuvre du "Bootstrap" : Intégration des bonnes pratiques	12
4.5 Évaluation théorique et bénéfices attendus pour l'INSEE	13
5 Étape 2 : L'Industrialisation de la CI/CD	15
5.1 Expression du besoin	15
5.2 Analyse des options	15
5.3 Solution retenue et architecture de l'usine logicielle	16
5.4 Implémentation du pipeline standardisé	17
5.5 Évaluation théorique : Fiabilité, conformité et valeur pour la DSI	17
6 Étape 3 : Le Déploiement Continu et l'Infrastructure	18
6.1 Expression du besoin : La complexité du déploiement et le défi de l'hétérogénéité des infrastructures	18
6.2 Panorama des technologies et étude des approches d'abstraction	19
6.3 Arbitrage : Backstage vs Kratix vs Crossplane	19
6.4 Architecture de déploiement cible : Le modèle GitOps avec GitLab CI et ArgoCD	21
6.5 Réalisation du PoC : L'alliance Backstage, Crossplane et ArgoCD	21
6.6 Bilan et valeur stratégique pour la DSI	22
7 Étape 4 : Le Point d'Accès Unique : L'Internal Developer Platform	23
7.1 Expression du besoin : Combattre la dispersion et créer la "Single Source of Truth"	23
7.2 Étude comparative des solutions de portail développeur	23
7.3 Le concept d'Internal Developer Platform (IDP)	24
7.4 Arbitrage : le choix de l'outil IDP	24
7.5 Choix et justification de Backstage comme IDP	25
7.6 Unification du Golden Path au sein du portail	26
7.7 Bilan et valeur ajoutée organisationnelle	27
Conclusion	29
8 Annexe 1	31
9 Annexe 2	32
10 Annexe 3	33
11 Annexe 4	34
12 Annexe 5	35

Introduction

Contexte global

Au cours des deux dernières décennies, la transformation numérique est devenue un enjeu stratégique majeur pour les administrations publiques, et plus particulièrement pour l'INSEE. Pour répondre aux exigences croissantes d'agilité, de résilience et de rapidité de livraison, les Systèmes d'Information (SI) se sont profondément modernisés. L'adoption progressive du Cloud, des architectures orientées microservices, des conteneurs et de la méthodologie DevOps a permis d'accélérer le rythme des déploiements tout en offrant une flexibilité importante.

Cependant, cette évolution s'est accompagnée d'une complexification systémique de l'écosystème technologique. À tous les niveaux de la chaîne de valeur, la multiplication des outils de développement, de déploiement, de sécurité et de supervision ont progressivement augmenté la charge cognitive des équipes applicatives. Au sein de la Direction des Systèmes d'Information (DSI) de l'INSEE, ce phénomène de dispersion des outils s'est traduit par l'apparition d'hétérogénéités, de rigidités opérationnelles et d'une perte d'efficacité globale, menaçant la promesse initiale d'agilité portée par la culture DevOps.

Contexte et cadre du stage

C'est dans ce contexte de refonte et de modernisation des pratiques d'ingénierie que s'inscrit ce stage de fin d'études, réalisé au sein de l'équipe KubeApp de la DSI. Mon travail vise à réinterroger les modes de collaboration et d'accès aux ressources informatiques de l'organisation, à la fois au niveau des équipes de développement applicatif et les équipes d'exploitation.

Afin d'apporter une réponse à ces enjeux, la DSI a initié une réflexion centrée sur le concept de **Platform Engineering**. Cette démarche vise à concevoir l'infrastructure et l'outillage interne non plus comme un ensemble de contraintes ou de tickets d'assistance, mais comme un véritable **produit** destiné aux développeurs. L'enjeu central est de proposer une expérience développeur (**Developer Experience - DevEx**) optimisée, capable d'allier autonomie des équipes et gouvernance de la DSI.

Objectifs du mémoire et démarche suivie

L'objectif principal de ce mémoire est de formaliser la conception et le déploiement d'une démarche de **Platform Engineering** adaptée aux spécificités et aux exigences de l'INSEE. Pour mener à bien cette mission, la démarche adoptée s'articule autour de trois objectifs majeurs :

1. **Comprendre et quantifier les limites de l'organisation actuelle** : À travers la réalisation d'un état des lieux et le traitement d'une enquête interne menée auprès des équipes techniques, il s'agit de cartographier précisément les irritants du quotidien et les goulots d'étranglement afin d'isoler les besoins prioritaires des utilisateurs.
2. **Étudier le paradigme du Platform Engineering et des portails développeur** : Réaliser une étude théorique et comparative des concepts de *Self-Service*, d'abstractions d'infrastructure et d'outils de portail développeur afin de définir le cadre de référence le plus pertinent pour la DSI.
3. **Concevoir et valider des parcours standardisés (Golden Paths)** : Élaborer des modèles opérationnels d'ingénierie, de l'initialisation automatisée d'un projet jusqu'à sa gestion en production, permettant de répondre directement aux besoins identifiés tout en garantissant une conformité native (*Compliance by Design*) aux exigences de sécurité, de qualité et d'urbanisation de l'organisation.

À travers cette étude, ce mémoire vise à démontrer comment une approche moderne orientée **Platform Engineering** permet de réduire la charge cognitive des développeurs, de lever la dépendance aux experts et d'offrir à la DSI une gouvernance claire, unifiée et pérenne de son patrimoine applicatif.

1 Environnement et État des Lieux

1.1 Organisation et équipe d'accueil

L'**Institut national de la statistique et des études économiques (INSEE)** est une direction générale du ministère de l'**Économie et des Finances**. Il a pour mission de collecter, analyser et diffuser des informations sur l'économie et la société française sur l'ensemble de son territoire.

En particulier, ce stage s'est déroulé au sein de la **Direction du Système d'Information (DSI)**, qui gère l'ensemble du Système d'Information (SI) de l'INSEE. Elle conçoit la stratégie numérique, garantit la sécurité des données et fournit les outils de travail adaptés aux différents acteurs de l'institut.

La DSI est composée de 3 entités centrales :

Le **Département du Développement du Système d'Information (DDSI)**, où on trouve les **Développeurs**

Les développeurs s'occupent de la conception, de l'implémentation et de la maintenance des applications, des logiciels et du site WEB de l'Insee (www.insee.fr). Chaque équipe de développement travaille avec sa maîtrise d'ouvrage (MOA) qui lui spécifie ses besoins.

Le **Département de la Production et de l'Infrastructure Informatiques (DPII)**, où on trouve les **Exploitants**, appelés **Ops** au sein de ce rapport

La production, quant à elle, gère la mise à disposition et le maintien des divers outils informatiques. Les travaux de production comprennent : la conception et maintenance du réseau interne, la gestion du parc bureautique, la conception et le maintien des infrastructures de production et déploiement, ainsi que la supervision des applications développées par le DDSI.

Le DPII pilote la production et prend les décisions concernant le système d'information de l'INSEE. Ce sont les divisions du pilotage qui sont maîtrises d'ouvrage des applications transverses à l'informatique (mail, annuaire...).

Deux services sont fonctionnellement rattachés au DPII :

- Les équipes du **Support National du Système d'Information (SNSI)** : Basés à Nantes, ils désignent les solutions techniques à mettre en oeuvre et offrent le support aux différentes technologies du système d'information actuel
- Les équipes du **Centre d'Exploitation Informatique (CEI)** : Ils sont en charge d'appliquer en production les gestes techniques validés par les équipes nantaises. Ils assurent également la supervision de la production et l'intégration des applications.

L'**Unité de l'Innovation et de la Stratégie du Système d'Information (UnISSI)**

Cette unité, entre le développement et la production, est composée de 3 divisions :

- **Division urbanisation du système d'information (DUSI)** : Responsable de l'urbanisation, de la cartographie du système d'information et de proposer des actions pour sa rationalisation métier
- **Division sécurité et maîtrise des risques (DSMR)** : En charge de définir les politiques de sécurité informatique de l'Insee, de pousser l'esprit sécurité auprès des agents et d'organiser les actions de remédiation en cas d'incident de sécurité
- **Division innovation et instruction technique (DIIT)** : Accompagne les démarches d'innovation au sein de l'Insee. Elle est également propriétaire et administratrice de la plateforme innovation, une infrastructure de type Cloud privé dédiée à la pratique d'activités innovantes à la fois informatiques, organisationnelles et statistiques

1.2 Existant informatique

Cadre de développement

À l'INSEE, le développement est très encadré par l'**Animation du Développement**, ce qui rend la stack technique applicative très uniforme :

- **Backend** : Standardisé autour de Java et du framework Spring Boot
- **Frontend** : Principalement structuré autour de React

Cette homogénéité applicative constitue un atout majeur pour la DSI : elle facilite la mobilité des développeurs entre les projets et fluidifie les échanges entre les équipes.

Écosystème d'infrastructure

Kubernetes et les services managés internes

L'infrastructure conteneurisée repose sur plusieurs clusters Kubernetes dédiés et cloisonnés selon les exigences de sécurité :

- **KubeProd** : Cluster réservé aux applications de production
- **KubePP** : Cluster dédié aux environnements de Pré-Production et de recette
- **KubeDMZ** : Zone démilitarisée pour les services exposés à l'extérieur du réseau interne

Autour de ces clusters, la DSI propose des services managés internes pour autonomiser les applications :

- **Stockage objet (S3)** : Basé sur la technologie **MinIO**, offrant un stockage sécurisé et consolidé pour les applications conteneurisées (KubeApp)
- **Base de données à la demande** : Offre de bases de données **PostgreSQL** directement intégrées et managées au sein de Kubernetes (avec des outils de sauvegarde comme PGBackRest et d'export/anonymisation de données comme PG2Parquet)
- **Volume persistant chiffré** : Sécurisation renforcée des données stockées au niveau des conteneurs

Infrastructure traditionnelle (VM) et les services système

Le monde virtualisé reste ancré pour le système et les bases historiques :

- **Virtualisation & Sauvegarde** : Migration progressive vers un serveur centralisé de sauvegarde Proxmox et gestion des externalisations via robot de sauvegarde, en remplacement de solutions propriétaires comme Cohesity
- **Échanges inter-applicatifs** : Les flux de données inter-quartiers (entre zones du SI) et les appels externes sont obligatoirement canalisés et sécurisés par l'API Manager **Gravitee**
- **Gestion des identités et secrets** : La sécurité des accès s'appuie sur **Keycloak** pour la gestion des identités (IAM / SSO) et sur **HashiCorp Vault** pour la centralisation sécurisée des secrets et mots de passe (complété par SPECOPS)

1.3 Pratiques DevOps

Si les outils DevOps sont bien implantés à la DSI, leur automatisation globale et la centralisation de la documentation restent des chantiers prioritaires.

L'usine logicielle et le modèle GitOps

L'usine logicielle s'appuie sur un socle d'outils éprouvés :

- **Forge & Intégration Continue (CI)** : **GitLab** assure la gestion des dépôts et l'exécution des pipelines de CI (**GitLab CI**)
- **Gestionnaire d'artefacts** : **Nexus** centralise les dépendances (jars, packages) et les images de conteneurs.
- **Déploiement Continu (CD)** : **ArgoCD** s'est imposé comme l'outil majeur de déploiement continu sur les clusters Kubernetes. En appliquant le modèle GitOps, ArgoCD garantit que l'état des clusters est toujours aligné avec les définitions stockées dans Git
- **Orchestration de tâches** : L'outil **Rundeck** est utilisé pour l'exécution programmée de jobs et de scripts d'exploitation système.

Infrastructure as Code (IaC) et Observabilité

- **Provisionnement et configuration** : La gestion de l'infrastructure évolue vers le un paradigme totalement déclaratif grâce au binôme **OpenTofu** (pour le provisionnement de ressources réseau, VM et Cloud) et **Ansible** (pour l'automatisation des configurations et des déploiements système)
- **Supervision et Observabilité** : La DSI opère une transition d'outils historiques (comme Centron) vers la suite **ELK** (**Elasticsearch**, **Logstash**, **Kibana**). Cette stack centralise les métriques réseau, la santé des services et l'analyse de logs provenant des équipements de sécurité et de routage (Palo Alto, Checkpoint, Cisco, F5).

La documentation

Malgré cet outillage moderne, l'enquête de maturité DevEx, que nous développerons dans la partie suivante, met en évidence un problème récurrent : la **dispersion des informations**. La connaissance est éparpillée entre plusieurs plateformes (Wikis, outils de ticketing, dépôts GitLab, documentation réseau/système).

2 Problématique et Limites du Modèle Actuel

L'analyse terrain menée au sein du DSI, à travers des interviews individuelles et des ateliers de focus groups, a permis de dresser un diagnostic précis de l'expérience développeur (DevEx) et du modèle d'exploitation en place. Si les démarches d'automatisation et de modernisation des infrastructures ont apporté des avancées significatives, elles ont également révélé des limites structurelles importantes.

2.1 Analyse de l'enquête interne : Irritants et points de friction

L'étude des retours utilisateurs met en évidence de nombreux points de friction qui freinent le quotidien des équipes de développement.

Dispersion des ressources et opacité de l'information

Le premier constat réside dans la difficulté d'accès aux ressources partagées. Les informations, guides et documentation technique sont dispersés sur de nombreux supports non synchronisés, souvent non référencés ou obsolètes. Il en résulte un manque de visibilité sur les composants disponibles, une sous-utilisation de la documentation et une grande difficulté à se coordonner entre les équipes applicatives.

Complexité de l'outillage et décalage avec le profil développeur

Les outils mis à disposition par les équipes de production sont souvent jugés inadaptés à l'expérience des développeurs. Ils sont perçus comme lents, complexes et très proches du métier d'exploitation (Ops), et imposent un jargon technique peu maîtrisé par les développeurs.

« Snapshot, chart, label, CVE... c'est incompréhensible » Extrait d'atelier

Ces difficultés poussent les développeurs à éviter certains outils ou à reproduire des schémas d'exécution sans compréhension réelle de ce qu'il se passe.

« Je fais du copié-collé » Extrait d'atelier

Dépendance critique aux experts et rétention du savoir

Les opérations liées à la production (provisionnement de bases de données, configuration CI/CD, déploiement Kubernetes) reposent beaucoup sur un nombre limité de profils qualifiés ("sachants"). Les développeurs moins expérimentés manquent d'autonomie et de supports adaptés pour monter en compétences, ce qui entraîne un recours systématique à ces profils.

« Je demande à un sachant » / « Je demande à ANONYME » Extraits d'atelier

Ce fonctionnement crée des goulots d'étranglement importants, ralentit les cycles de livraison et sollicite excessivement les experts, les détournant de leurs missions initiales.

Déficit de coordination et gouvernance des évolutions transverses

Les évolutions poussées par les équipes transverses (Infrastructures, Sécurité, Exploitation) font preuve d'un manque de vision globale. Elles sont diffusées sur de nombreux canaux mal identifiés, ce qui fait que les nouvelles exigences (mises à jour de sécurité, migrations d'infrastructure, ...) sont souvent appliquées de manière descendante, sans prise en compte des feuilles de route (backlogs) des équipes produit.

« On veut bien participer aux tâches, mais prenez en compte nos backlogs » Extrait d'atelier

« Les incidents ça arrive, mais communiquez. » Extrait d'atelier

Opacité, lenteur des pipelines et intégration tardive de la sécurité

Les pipelines de build et de déploiement sont perçus comme des "boîtes noires" lentes et capricieuses. La complexité du ciblage des erreurs et la lenteur des runners génèrent une perte de confiance dans l'infrastructure logicielle, incitant parfois certains développeurs à regretter des modes de déploiement manuels plus artisanaux mais perçus comme plus prévisibles.

« Pourquoi c'est lent ? Pourquoi ça plante ? Ok c'est cassé mais dites-le-moi ! » Extrait d'atelier

De plus, l'intégration des contrôles de sécurité (scans de vulnérabilités, validations d'architecture) intervient souvent en fin de chaîne, de manière bloquante. La sécurité est alors perçue comme un obstacle plutôt que comme un accompagnement.

2.2 Synthèse des besoins exprimés par les équipes

Pour répondre à ces irritants, les équipes de développement ont formulé un ensemble de besoins clairs. Ceux-ci se classent en deux catégories : les **Besoins cœur** indispensables à la constitution de la plateforme, et les **Besoins d'accélération** destinés à optimiser les différents processus du DSI.

Les Besoins Cœur

1. **Point d'entrée unique (Developer Portal) :** Centralisation de la documentation, de l'état des environnements, des liens applicatifs et du catalogue d'outils via une interface claire dotée d'un moteur de recherche.
2. **Self-service généralisé :** Capacité d'exécuter en autonomie les opérations récurrentes (création d'un namespace, provisionnement de bases de données, création d'un projet, ...).
3. **Standardisation des pratiques :** Mise à disposition de modèles prêts à l'emploi (templates CI/CD, squelettes de code) intégrant nativement les règles de sécurité et de versioning poussées par les équipes transverses (Architecture, Urbanisation, Sécurité)
4. **Observabilité unifiée :** Synthèse de l'état des services, des logs, des alertes et des pipelines au sein d'un tableau de bord unique pour accélérer le diagnostic lors des incidents.

Les Besoins d'Accélération

1. **Accompagnement et Onboarding :** Mettre en place des parcours guidés (docs-as-code, ...) et du mentorat pour accélérer l'intégration des nouveaux arrivants et réduire le temps d'apprentissage.
2. **Communication et visibilité organisationnelle :** Canal unique pour les annonces d'incidents ou d'évolutions, accompagné d'un annuaire identifiant clairement les responsabilités et les experts par domaine.
3. **Optimisation des performances de la CI/CD :** Réduction des temps de build par la mise en cache, la parallélisation et un feedback immédiat sur les erreurs.

2.3 Conséquences organisationnelles et techniques

Les dysfonctionnements et les besoins non couverts observés à l'échelle opérationnelle ont des répercussions directes sur la performance globale de l'INSEE.

Dégradation du Time-to-Market

L'allongement des cycles de livraison constitue l'impact le plus visible. La validation tardive des contraintes de sécurité et d'architecture crée des itérations coûteuses juste avant la mise en production. De plus, à cause de la dispersion de la documentation et de la lenteur des pipelines, ce fonctionnement ralentit la capacité de la DSI à livrer de la valeur métier à ses utilisateurs.

Augmentation du risque opérationnel et accumulation de dette technique

Le rejet des outils perçus comme trop complexes conduit à une divergence des pratiques. Bien que les technologies de développement soient assez homogènes (Java / SpringBoot), chaque équipe a tendance à développer ses propres scripts et configurations locales, que ce soit au niveau de la CI ou au niveau du déploiement, ce qui les rend très difficiles à maintenir dans le temps. Ce phénomène s'accompagne d'un risque élevé lié au coût du Run : la présence de plusieurs technologies non standardisées surcharge les équipes d'exploitation qui doivent maintenir plusieurs générations de stacks technologiques.

Incapacité à absorber la croissance à l'échelle

Dans le modèle actuel, l'arrivée d'un nouveau projet exige une multitude d'interventions manuelles et de validations croisées. Les équipes transverses consacrent une majeure partie de leur temps de travail au maintien en condition opérationnelle (Run) et au traitement de tickets récurrents, au détriment des activités de construction (Build) et d'innovation.

Coûts cachés et perte de productivité

Enfin, ce contexte génère des coûts invisibles mais réels :

- **Gaspillage du temps ingénieur :** Des heures d'ingénierie à forte valeur ajoutée sont consommées dans des tâches de coordination, de recherche d'information ou d'attente.
- **Charge cognitive :** La frustration accumulée face aux blocages techniques et à la complexité administrative dégrade la motivation des développeurs et augmente leur fatigue mentale en créant une surcharge cognitive.

3 Le Platform Engineering : Une Approche Centrée Produit

Le DevOps a révolutionné l'industrie logicielle en brisant *le mur de la confusion* entre les développeurs (Dev) et les exploitants (Ops). Cependant, la mise en œuvre brute de cette nouvelle organisation a fini par se heurter à des limites structurelles. Le Platform Engineering ne remplace pas le DevOps, mais s'impose comme son évolution mature, une réponse organisationnelle et technique conçue pour rationaliser l'expérience des ingénieurs au sein des systèmes d'information modernes.

3.1 Origine et évolution

Le mouvement DevOps reposait sur une promesse forte : "You build it, you run it" (Vous le concevez, vous le gérez). En responsabilisant les développeurs sur l'ensemble du cycle de vie de leurs applications, du code initial jusqu'à la production, l'INSEE a pu considérablement accélérer sa vitesse de livraison. En particulier, cette autonomie a permis de supprimer les frictions liées aux passages de relais incessants entre équipes.

Cependant, le SI de l'INSEE a vu son écosystème technologique exploser en complexité. L'avènement du Cloud Native, la généralisation des architectures de microservices, la conteneurisation via Kubernetes, l'Infrastructure as Code (IaC) ou encore les exigences accrues en matière de sécurité (DevSecOps) ont radicalement modifié le quotidien des équipes applicatives.

Pour livrer une simple fonctionnalité en production, un développeur ne peut plus se contenter de maîtriser son langage de programmation. Il doit désormais comprendre :

- Les manifestes Kubernetes et le routage réseau.
- La configuration de pipelines de CI/CD et de scanners de vulnérabilités.
- L'écriture de scripts Terraform ou OpenTofu pour provisionner ses ressources Cloud.
- La mise en place de politiques de monitoring, d'alerting et de gestion des logs.

Cette accumulation de responsabilités a engendré un phénomène critique : l'explosion de la charge cognitive. Au lieu de se concentrer sur la logique métier, là où réside la véritable valeur ajoutée pour l'INSEE, les développeurs passent une part de plus en plus importante de leur temps à se débattre avec l'infrastructure informatique. Face à cette surcharge, le modèle "You build it, you run it" s'est souvent transformé en "You build it, and you run away". C'est pour éviter cette dérive et redonner de l'efficacité aux équipes que le Platform Engineering a émergé.

3.2 Les piliers du Platform Engineering

Le Platform Engineering se définit comme la discipline consistant à concevoir et à construire des plateformes internes en libre-service afin d'améliorer l'expérience développeur (DevEx) et d'accélérer la livraison de valeur. Cette approche repose sur quatre piliers fondamentaux :

L'approche "Platform as a Product" (La plateforme comme un produit)

Il s'agit du changement de paradigme le plus crucial. Une plateforme interne ne doit pas être un projet technique imposé par la hiérarchie, mais un produit dont les utilisateurs finaux sont les développeurs de l'INSEE. L'équipe en charge de la plateforme (l'équipe Plateforme) adopte la posture d'un éditeur de logiciel : elle nomme un Product Owner, mène des enquêtes internes, recueille les feedbacks, mesure l'adoption et gère un backlog de fonctionnalités afin d'améliorer la plateforme en continu. On ne force alors pas les développeurs à utiliser la plateforme, on crée plutôt une plateforme suffisamment performante pour qu'ils choisissent eux-mêmes de l'adopter.

L'atténuation de la charge cognitive

Le but premier de la plateforme est de masquer la complexité inutile de l'infrastructure sans pour autant la faire disparaître. La plateforme agit donc comme une couche d'abstraction : elle encapsule l'expertise des équipes Ops dans des outils ou des APIs, permettant aux développeurs d'interagir avec l'infrastructure sans avoir besoin d'en maîtriser l'implémentation, ce qui leur permet de se concentrer sur leur cœur de métier et de réduire leur charge cognitive.

Le Self-Service (Libre-service)

Pour éliminer les tickets d'assistance ("Ouvrir un ticket pour obtenir une base de données", ...), la plateforme doit être entièrement automatisée et accessible en autonomie. Le développeur doit pouvoir instancier ses besoins de manière simple, éventuellement via une interface graphique, et instantanée sans dépendre d'une validation humaine manuelle. Cette automatisation a un intérêt double : Fluidifier et accélérer l'expérience développeur, et alléger le quotidien des exploitants en supprimant de nombreux tickets répétitifs.

La standardisation par défaut

En centralisant la création des composants, la plateforme applique nativement les standards internes. Qu'il s'agisse de la configuration des rôles IAM, de l'architecture des fichiers de code ou encore de la mise en place de la pipeline CI/CD, la sécurité et la conformité sont intégrées d'office à la création de chaque ressource. Les développeurs se munissent alors d'un cadre clair et sécurisé dès le début de leur projet.

3.3 Acteurs et Personas : À qui s'adresse la plateforme

Concevoir la plateforme interne comme un produit impose de définir clairement à qui elle s'adresse et quels problèmes elle cherche à résoudre pour chaque catégorie d'utilisateurs. Une plateforme n'est pas qu'un outil réservé aux développeurs, elle crée de la valeur pour l'ensemble de la chaîne du SI, des équipes de réalisation jusqu'au management.

Les trois personas cibles de la plateforme

Afin de concevoir des interfaces et des parcours adaptés, on a identifié trois personas à l'aide de l'enquête de terrain réalisée au sein de la DSI. Ces personas ont pour objectif de définir les utilisateurs types de notre plateforme pour mieux cibler les besoins auxquels devra répondre la plateforme. Ces 3 personas sont :

- **Le Consommateur** : Il s'agit principalement du développeur applicatif au quotidien. Son objectif principal est d'écrire du code métier et de le livrer rapidement en production. Il ne souhaite pas avoir à appréhender la complexité de l'infrastructure (fichiers YAML complexes, règles IAM, réseau, ...). Ce persona est la cible principale des Golden Paths et des fonctionnalités en libre-service (self-service). Pour lui, la plateforme doit offrir une expérience fluide où la création d'un projet ou le déploiement d'un service s'effectue en quelques clics ou via une simple commande.
- **Le Contributeur** : Ce profil regroupe les développeurs seniors, les architectes, mais aussi les ingénieurs DevOps et SRE. En plus de l'automatisation, il a besoin de garder le contrôle, de comprendre les mécanismes sous-jacents et parfois de sortir du cadre standard pour des besoins spécifiques. Le Contributeur, en plus de consommer la plateforme, participe à son enrichissement. Il est la cible des interfaces avancées (CLI, APIs brutes, templates modifiables) et contribue activement au catalogue interne en créant de nouveaux modèles (scaffolding) ou de nouvelles briques d'infrastructure.
- **Le Manager / Tech Lead** : Ce persona regroupe les responsables d'équipe, les Tech Leads et les responsables de la sécurité. Il n'intervient pas directement dans la création de ressources au quotidien, mais a besoin d'une vision claire et transverse du SI. Il est la cible des tableaux de bord (dashboards), du catalogue de services et des indicateurs de conformité. La plateforme lui apporte la visibilité nécessaire : état de santé des applications, niveau de dette technique, respect des standards de sécurité et cartographie des dépendances.

Bénéfices attendus

Le Platform engineering, bien que pensé pour les développeurs, présente en réalité des avantages à la fois pour les développeurs, mais aussi pour les exploitants :

- **Pour les Développeurs** : Ils gagnent en autonomie : La suppression des délais d'attente liés au traitement manuel des demandes d'infrastructure (ouverture de ports, création de bases de données, génération de pipelines, ...) élimine le sentiment de frustration et réduit considérablement les temps de livraison. Par ailleurs, leur charge mentale est allégée : Les développeurs bénéficient désormais d'une infrastructure à l'état de l'art de la DSI sans avoir à maîtriser les outils sous-jacents.
- **Pour les Exploitants** : La plateforme les libère de la gestion répétitive des tickets simples de niveau 1. Au lieu de provisionner manuellement des ressources pour chaque projet, les Ops codent les règles, la sécurité et la conformité directement dans la plateforme. Ils passent ainsi d'un rôle réactif de "support" à un rôle d'"ingénieurs plateforme", concentrant leur expertise sur la scalabilité, la résilience de l'infrastructure globale et l'amélioration continue du produit.

3.4 Le concept de Golden Path (Le "Chemin Pavé")

Popularisé par les équipes de Spotify, le concept de **Golden Path** (ou *Golden Road*) est l'application concrète des piliers du Platform Engineering. On le définit comme le chemin balisé, documenté et soutenu par l'organisation pour accomplir une tâche ou construire un logiciel.

Le Golden Path est un ensemble d'outils, de modèles de code (templates), de pipelines automatisés et de services intégrés qui permettent de passer de l'idée à la production de la manière la plus fluide possible. Si un développeur choisit de suivre ce "chemin pavé", il bénéficie :

- D'une vitesse d'exécution maximale (le provisionnement est réalisé de quelques minutes).

- D’une conformité automatique avec les exigences de sécurité et d’architecture de la DSI.
- D’un support complet de la part de l’équipe Plateforme en cas d’incident.

Compromis : Liberté vs Cadre

On pourrait penser que le Golden Path est une prison rigide qui prive les développeurs de leur liberté de choix technique. En réalité, c’est le contraire : le Platform Engineering prône une approche de liberté responsable.

L’organisation n’interdit pas aux équipes de sortir du Golden Path. Si un projet ou un besoin métier très spécifique nécessite un type de base de données non standard, un langage de programmation non prévu, etc., l’équipe de développement a toujours le droit de sortir du Golden Path. Cependant, l’équipe qui s’éloigne des standards doit assumer elle-même la maintenance, la sécurité, le requêtage et le support de son infrastructure spécifique, sans pouvoir solliciter l’aide prioritaire de l’équipe Plateforme.

Le Golden Path crée ainsi des garde-fous plutôt que des barrières. Il incite à la standardisation par la valeur et le confort plutôt que par la contrainte ou l’interdiction. Pour la majorité des projets internes, le Golden Path représente la voie de la simplicité, permettant de maximiser l’efficacité globale.

3.5 Comment la plateforme répond-elle à nos besoins

Douleurs & Irritants	Besoins exprimés	Piliers du Platform Engineering
Dispersion des ressources, doc obsolète & fragmentation	Point d’entrée unique & Accès à la connaissance	Portail Développeur (Software Catalog, TechDocs)
Dépendance critique aux experts Ops & goulots d’étranglement	Autonomie via le <i>Self-Service</i> généralisé	Golden Path & Self-Service (Scaffolder, Templates de code standardisés)
Complexité du jargon Ops (Kube, YAML) & surcharge cognitive	Standardisation, masquage de la complexité infra	Abstraction & Infrastructure-as-Code
Opacité, lenteur de la CI/CD & détection tardive des CVE	Optimisation CI/CD, feedback rapide & sécurité native	Usine logicielle & DevSecOps (GitLab CI, SonarQube, etc. mutualisés)
Déploiements manuels/artisans & opacité de la Prod	Observabilité unifiée & lisibilité de l’état des services	GitOps & Orchestration de Déploiement (ArgoCD, Grafana, etc. mutualisés)
Déficit de coordination transverse & intégration difficile des exigences	Gouvernance claire, Onboarding guidé & communication unifiée	Product Management pour la Plateforme (Pratiques DevEx, gouvernance par l’Animation Dev)

TABLE 1 – Alignement des irritants, des besoins et des piliers du Platform Engineering

Ce tableau met en évidence l’alignement direct entre les irritants observés sur le terrain et la réponse apportée par le **Platform Engineering**. Pour concrétiser cette vision théorique et répondre aux attentes des équipes applicatives, la suite de ce mémoire est structurée autour de quatre étapes opérationnelles majeures, constituant la mise en œuvre pratique de notre **Golden Path** :

1. **L’initialisation et la création de projets** : La mise en place de modèles dynamiques (**Scaffolder**) pour garantir un démarrage immédiat en libre-service, respectant nativement les standards de sécurité et de qualité dès la création du projet (**Compliance by Design**).
2. **L’intégration continue et la qualité de code** : L’unification de l’usine logicielle et des pipelines de CI/CD afin d’automatiser les contrôles de sécurité, d’accélérer les temps de feedback et de supprimer la maintenance individuelle des scripts de build.
3. **Le déploiement continu et la gestion de l’infrastructure** : L’adoption du modèle GitOps et la mise en œuvre d’abstractions d’infrastructure visant à masquer la complexité de Kubernetes et du Cloud Native, offrant aux développeurs un déploiement fluide et prévisible.
4. **Unification au sein d’un portail développeur (IDP)** : La centralisation de l’ensemble de ces briques au sein d’un point d’accès unifié (Backstage), offrant un catalogue de services dynamique, une documentation intégrée (TechDocs) et une observabilité globale du patrimoine applicatif.

4 Étape 1 : Initialisation et Création de Projets

La première étape de notre Golden Path concerne l'initialisation d'un projet. Au sein d'un projet, c'est à ce moment là que se posent les questions techniques de l'application et que se détermine sa conformité future avec les exigences internes.

Une gestion défaillante de cette phase a des répercussions qui se prolongent sur toute la durée du projet, en conditionnant sa maintenabilité et sa conformité pour l'ensemble de son cycle de vie.

4.1 Expression du besoin

À l'INSEE, la création d'un nouveau projet représentait une source importante de friction et d'hétérogénéité. Lorsqu'une équipe devait développer un nouveau service, malgré un bon encadrement des pratiques de développement au sein de la DSI, elle se heurtait immédiatement à deux obstacles majeurs :

- **Le syndrome de la "page blanche"** : Partir de zéro implique de devoir créer l'arborescence du projet, choisir les dépendances, configurer les fichiers de builds, structurer le code métier, etc.. Cette phase, en plus d'être à faible valeur ajoutée pour les développeurs, est chronophage, ce qui peut provoquer du retard dans la livraison des premières fonctionnalités.
- **La divergence par rapport aux standards** : Sans outil d'initialisation automatisé ou de documentation unifiée, chaque équipe applique ses propres conventions (dépendances, structuration des dossiers, politiques de logs, gestion des configurations, ...). De plus, la pratique courante du "copier-coller" (dupliquer un projet existant "proche") entraîne la propagation invisible de dette technique, de configurations obsolètes ou de failles de sécurité.

Au-delà de la perte de temps individuelle, cette absence de cadre commun génère un coût caché mais bien réel pour l'organisation : chaque projet devient un cas particulier, ce qui complexifie la supervision transverse, alourdit la charge de maintenance à long terme et retarde la détection des non-conformités.

Le besoin exprimé par les développeurs est donc clair : disposer d'un mécanisme capable d'**instancier en quelques clics un projet prêt à coder**, configuré nativement selon l'état de l'art de la DSI, sans nécessiter d'interventions manuelles répétitives.

4.2 Étude comparative des solutions

Pour répondre à ce besoin de création de projet, plusieurs paradigmes d'ingénierie logicielle existent sur le marché.

Cette analyse a été conduite en confrontant chaque approche à trois critères jugés prioritaires pour l'INSEE : la vitesse offerte aux équipes de développement, le niveau de gouvernance et de conformité garanti, et la capacité de la solution à s'intégrer dans l'écosystème plus large de la plateforme (catalogue, documentation, CI/CD).

La copie de dépôt existant : Le "Copier/Coller"

C'est la méthode la plus intuitive mais aussi la plus risquée. Un développeur clone un projet existant et supprime le code métier pour n'en garder que la structure.

- **Avantages** : Ne nécessite aucun investissement technique ou outillage spécifique.
- **Inconvénients** : Propagation de la dette technique, risques d'oubli de nettoyage de secrets/clés d'API, dépendances dépassées et absence totale de gouvernance globale.

Les modèles intégrés aux forges logicielles (GitLab / GitHub Templates)

Les forges logicielles permettent de définir des dépôts "modèles". Lors de la création d'un projet, la forge (dans notre contexte : GitLab) recopie l'arborescence du modèle.

- **Avantages** : Simplicité d'administration au niveau de la forge, très bonne intégration dans l'écosystème Git.
- **Inconvénients** : Modèles statiques. Ils ne permettent pas d'injecter facilement de la logique dynamique lors de la création (ex : paramétrer dynamiquement le nom du service, générer des identifiants uniques, interagir avec d'autres systèmes de la DSI comme créer un projet SonarQube ou une entrée dans le catalogue).

Le "Scaffolding" dynamique via un générateur d'écosystème

Le Scaffolding repose sur l'utilisation d'un moteur de modèles dynamiques capable non seulement de générer du code sur mesure à partir de variables saisies par l'utilisateur (nom, variables d'environnement, type d'infrastructure, ...), mais aussi d'orchestrer des actions annexes dans le SI via des workflows automatisés.

- **Avantages** : Dynamisme complet, intégration native dans le portail développeur, capacité à orchestrer la création du projet Git, la configuration des accès et l'enregistrement dans le catalogue de services en une seule opération.
- **Inconvénients** : Nécessite un certain apprentissage de la syntaxe des templates et de la maintenance du moteur.

Cette comparaison met en évidence un arbitrage classique en ingénierie logicielle : les solutions les plus simples à mettre en œuvre (copier/coller, modèles de forge) sont aussi celles qui offrent le moins de garanties sur le long terme, tandis que l'investissement initial que représente le Scaffolding dynamique est rapidement compensé par les gains de gouvernance et d'homogénéité qu'il procure à l'échelle de l'organisation.

4.3 Solution retenue et architecture

Notre choix s'est naturellement porté sur le Scaffold de Backstage (nous évoquerons Backstage plus en détail dans la section 7), testé et adapté sous forme de preuves de concept (POC) pour nos usages internes.

Ce choix est motivé par la vision "Produit" de notre plateforme : au lieu de multiplier les outils, le Scaffold permet au développeur de rester dans le même environnement (Backstage) pour consulter la documentation, découvrir des services et créer son projet.

D'un point de vue macroscopique, l'architecture du composant s'articule autour de trois éléments clés :

- **L'Interface Utilisateur (Formulaire dynamique)** : Un questionnaire simple présenté dans le portail Backstage demandant le nom du composant, la description, l'équipe propriétaire et les options techniques souhaitées.
- **Le Moteur d'Exécution** : Il récupère le squelette de code hébergé, remplace les variables (variables d'environnement, noms de packages, ...) et génère la structure personnalisée.
- **Les Connecteurs (Actions)** : Une fois le code généré, le Scaffold communique de manière transparente avec la forge logicielle pour créer le dépôt Git, y pousser le premier commit, et enregistrer automatiquement la nouvelle entité dans le catalogue.

Cette séparation entre l'interface, le moteur d'exécution et les connecteurs présente un intérêt organisationnel au-delà de la simple élégance technique : elle permet à l'équipe plateforme de faire évoluer chaque brique indépendamment (ajouter un champ au formulaire, changer de forge logicielle, brancher un nouvel outil du SI) sans remettre en cause l'ensemble du dispositif, ce qui garantit sa pérennité face aux évolutions futures de l'organisation.

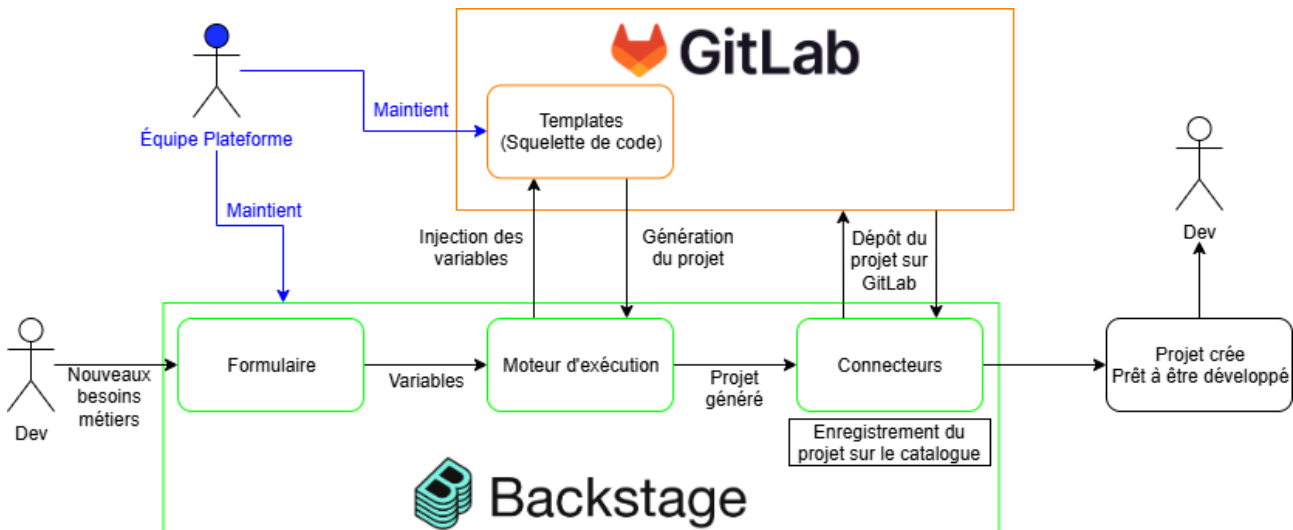


FIGURE 1 – Point de vue macroscopique du composant d'initialisation de projet

Des captures d'écran sont disponibles dans l'annexe 8.

4.4 Mise en œuvre du "Bootstrap" : Intégration des bonnes pratiques

La valeur du Scaffold réside dans la qualité des modèles (templates) mis à disposition. Notre travail s'est concentré sur la création d'un template type représentant le "Golden Path" pour une application interne standard (Java / SpringBoot + React).

C'est justement ce travail qui distingue un Golden Path d'un simple outil d'automatisation technique : la valeur de notre Golden Path ne réside pas dans la génération de code en elle-même, mais dans le fait que ce code embarque, dès sa création, l'ensemble des standards que l'organisation attend de ses applications.

En utilisant ce template, le développeur n'obtient pas un simple dépôt vide, mais une structure applicative immédiatement opérationnelle qui intègre nativement :

- **La structure de code standardisée** : Respect des conventions d'architecture recommandées par nos équipes transverses, facilitant la lisibilité et la reprise du code par d'autres développeurs.
- **La configuration de sécurité embarquée** : Exclusion native des fichiers sensibles (.gitignore pré-configuré), présence de politiques de gestion des secrets et intégration des mécanismes d'authentification internes.
- **L'usine logicielle pré-câblée** : Présence immédiate du fichier de pipeline de CI/CD (ex : .gitlab-ci.yml), préconfiguré pour exécuter les contrôles de qualité et de sécurité standards dès le premier push.
- **La documentation "As Code"** : Génération automatique d'un fichier README.md enrichi et des squelettes de documentation (Architecture, API) exploitables directement par l'outil de documentation centralisé du portail (TechDocs).

Grâce à ce "Bootstrap", l'équipe de développement passe d'une étape de configuration initiale lourde et sujette aux erreurs à un état où son unique préoccupation devient l'écriture des fonctionnalités métier.

4.5 Évaluation théorique et bénéfices attendus pour l'INSEE

Bien que le déploiement global de ce Golden Path d'initialisation n'ait pas encore fait l'objet d'une campagne de retours utilisateurs en production à grande échelle, l'expérimentation menée à travers nos prototypes permet de dégager **l'alignement parfait** de cette solution avec les besoins stratégiques des développeurs et de la DSI.

Ces bénéfices s'articulent autour de trois angles complémentaires : l'expérience quotidienne des développeurs, la gouvernance portée par la DSI, et la soutenabilité de la solution dans la durée.

Réponse aux besoins des développeurs (DevEx)

Pour les équipes applicatives, la valeur apportée se mesure en gain de sérénité et d'efficacité. La suppression de la phase de paramétrage initial (structure du projet, choix des dépendances, initialisation de la CI (ce dernier point sera abordé dans la section suivante)) réduit considérablement la charge cognitive au lancement d'un service. Le développeur n'a plus besoin d'être un expert des configurations complexes de la DSI pour démarrer dans les règles de l'art : le système guide ses choix via le portail et lui fournit un cadre sécurisant.

Réponse aux exigences de l'INSEE et gouvernance DSI

Pour la DSI et les équipes transverses, le Scaffold agit comme un levier de standardisation passif. Plutôt que de contrôler la conformité du code ou des configurations a posteriori via des audits coûteux, la conformité est garantie par construction (*Compliance by Design*).

En assurant que chaque nouveau projet naisse avec les bons outils de sécurité, des dépendances à jour et une documentation structurée, l'INSEE s'assure d'une réduction drastique de sa dette technique initiale et facilite grandement le Maintien en Condition Opérationnelle (MCO) de son parc applicatif.

Cette approche préventive s'inscrit dans une logique de maîtrise des risques : il est structurellement moins coûteux de garantir la conformité dès la création d'un projet que de la corriger a posteriori sur un existant déjà en production.

Simplification des évolutions et de la maintenance (« Day 2 »)

Au-delà de l'initialisation du projet, l'adoption de ce Golden Path apporte une valeur considérable sur la gestion du cycle de vie applicatif à long terme, communément appelée les évolutions « Day 2 ».

La standardisation des structures de projets et l'alignement sur un socle commun permettent aux équipes transverses de diffuser des mises à jour majeures (migrations de versions de Java/Spring Boot, correctifs de sécurité critiques ou évolutions des règles de CI/CD) de manière centralisée et homogène. Pour le développeur, l'effort de maintenance continu s'en trouve nettement allégé et facilité.

Ce bénéfice est d'autant plus significatif que le nombre de projets créés via le Golden Path augmente : chaque nouveau service homogène renforce l'effet de levier des équipes transverses, alors qu'à l'inverse, un parc hétérogène verrait le coût de chaque campagne de mise à jour croître avec la diversité des configurations à gérer.

Sans le Golden Path	Avec le Golden Path (Scaffolder Backstage)
Syndrome de la page blanche / Copie risquée : Création manuelle de l'arborescence ou clonage d'un projet existant avec suppression manuelle du code métier.	Instanciation dynamique en quelques clics : Saisie d'un formulaire simple (nom, équipe, options) générant immédiatement une arborescence sur-mesure.
Propagation de la dette technique : Risque élevé d'importer des dépendances obsolètes, des fichiers inutiles ou des erreurs de configuration masquées.	Socle technique et dépendances à l'état de l'art : Injection native des versions recommandées et des conventions d'architecture définies par la DSI.
Sécurité initiale manuelle et faillible : Oubli fréquent du '.gitignore', risque de fuite de clés/secrets hérités du projet copié, authentification à configurer de zéro.	Sécurité native (<i>Security by Design</i>) : Exclusion systématique des fichiers sensibles, intégration native des briques d'authentification et politiques de secrets.
Configuration CI/CD à rédiger : Rédaction manuelle ou copier-coller complexe du fichier de pipeline (ex : '.gitlab-ci.yml'), sujet à des erreurs de syntaxe.	Usine logicielle pré-câblée : Fichier de pipeline CI/CD généré et prêt à exécuter les scans de sécurité et de qualité dès le premier <i>commit</i> .
Projet isolé et non référencé : Création manuelle du dépôt Git, documentation souvent absente ou rédigée tardivement sur un Wiki externe.	Visibilité et enregistrement automatique : Dépôt Git créé automatiquement, service enregistré dans le Catalogue et squelette de documentation <i>TechDocs</i> prêt à coder.
Bilan : Phase d'initialisation longue (plusieurs heures/-jours), chronophage et source d'hétérogénéité.	Bilan : Projet prêt à coder en 2 minutes, conforme aux exigences DSI et libéré de la charge cognitive initiale.

TABLE 2 – Comparatif du processus d'initialisation de projet : Sans vs Avec le Golden Path (Scaffolder)

5 Étape 2 : L'Industrialisation de la CI/CD

Une fois le projet créé et structuré via le Scaffolder, l'étape suivante du Golden Path consiste à automatiser la validation et la transformation du code source en un artefact exécutable et sécurisé. La CI/CD (Intégration Continue / Déploiement Continu) constitue le moteur de cette usine logicielle. Ainsi, si l'étape précédente garantit qu'un projet naît conforme, celle-ci garantit qu'il le reste à chaque modification.

5.1 Expression du besoin

Historiquement, l'élaboration des pipelines de CI/CD au sein de l'INSEE relevait d'une démarche essentiellement artisanale. Chaque équipe projet écrivait ses propres scripts de build, de test et de packaging de manière isolée au sein de son dépôt de code.

Cette approche décentralisée souffrait de quatre défauts critiques :

- **La réinvention permanente de la roue** : Des dizaines d'équipes consacraient du temps à écrire et corriger des scripts similaires pour compiler du code, construire une image de conteneur ou exécuter des tests unitaires.
- **L'hétérogénéité et la dette technique** : En l'absence de socle commun, les pratiques de build divergeaient fortement d'un projet à l'autre. La maintenance de ces scripts customisés s'avérait lourde, notamment lors des montées de version des outils ou des langages.
- **L'intégration tardive ou absente de la sécurité** : Les contrôles de sécurité (scans de vulnérabilités, analyse statique de code) étaient souvent omis ou configurés de façon superficielle (souvent en `"allow_failure : true"` en pratique), faute de compétences spécifiques au sein des équipes applicatives.
- **L'absence de gouvernance globale** : Pour la DSI, et plus particulièrement la DSMR, il était quasiment impossible d'auditer le niveau de conformité ou de sécurité de l'ensemble des applications sans passer en revue chaque dépôt individuellement.

Ces défauts, mis bout à bout, dessinent un risque systémique pour l'INSEE, où le niveau de sécurité réel du parc applicatif dépend davantage de la bonne volonté de chaque équipe que d'une politique appliquée uniformément.

L'objectif était donc clair : industrialiser la CI/CD pour la transformer en un service clé en main, automatisé, maintenable et sécurisé par défaut.

5.2 Analyse des options

Pour répondre à ce besoin d'industrialisation, l'écosystème propose plusieurs approches et outils, chacun répondant à des philosophies d'organisation différentes.

Comme pour l'initialisation de projet, cette analyse a été guidée par trois critères : la simplicité d'adoption pour les développeurs, la capacité à mutualiser l'effort de maintenance entre projets, et le niveau de gouvernance atteignable sans surcharger les équipes transverses.

L'approche serveur central historique (ex : Jenkins)

Repose sur un serveur maître gérant l'exécution de pipelines écrits souvent sous forme de scripts complexes.

- **Avantages** : Historiquement très répandu, grand écosystème de plugins.
- **Inconvénients** : Lourdeur d'administration, maintenance de l'infrastructure centralisée complexe, syntaxe souvent éloignée du quotidien des développeurs et faible intégration "native" sous forme de Pipeline-as-Code.

Les forges modernes et leurs écosystèmes (ex : GitHub Actions, GitLab CI)

Approche basée sur le Pipeline-as-Code où la configuration est versionnée directement avec le projet dans des fichiers déclaratifs (YAML).

- **Avantages** : Proximité immédiate avec le code, simplicité de prise en main, excellente intégration dans l'expérience développeur.
- **Inconvénients** : Risque de duplication si chaque projet écrit son propre YAML sans réutiliser de modules centralisés.

L'évolution interne : Des "Components" faits maison vers un framework communautaire

Au sein de la DSI, où GitLab est la forge de référence, une première initiative interne avait vu le jour : la création de GitLab Components par l'équipe KubeApp. Ces composants peuvent être inclus de manière très simple et concise au sein de la CI, permettant ainsi de masquer la complexité sous-jacente des jobs de la CI. Bien que fonctionnels, ces composants développés "à partir de zéro" restaient rudimentaires et exigeaient un effort de maintenance important de la part de l'équipe pour suivre les évolutions technologiques.

Dans le cadre de ce stage, nous avons exploré une approche alternative à forte valeur ajoutée : le projet open-source "To Be Continuous" (TBC). Il s'agit d'un catalogue de composants GitLab CI complets, maintenus par une vaste communauté, prêts à l'emploi et respectant scrupuleusement l'état de l'art du Cloud Native et du DevSecOps.

Ce constat rejoint l'arbitrage identifié en amont : plutôt que de continuer en interne le développement d'un outillage que la communauté open-source entretient déjà à plus grande échelle, il devient plus pertinent pour l'INSEE de s'appuyer sur cet effort mutualisé et de recentrer ses ressources sur l'intégration et la gouvernance de ces briques.

```
maven-build:
  stage: "🔨 Build"
  script:
    - mvn $MAVEN_CLI_OPTS clean verify -DskipTests
  artifacts:
    paths:
      - target/*.jar
  cache:
    key: "$CI_COMMIT_REF_SLUG"
    paths:
      - .m2/repository
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

maven-test:
  stage: "🧪 Tests"
  script:
    - mvn $MAVEN_CLI_OPTS clean verify
  artifacts:
    expire_in: 1 week
    when: always # permet d'avoir le rapport même quand les tests échouent
    paths:
      - ./rapport-cucumber.html
      - ./target/ROOT.jar
    reports:
      junit: ./target/junit.xml
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

# Components
include:
# To Be Continuous : Maven Component (alors peut etre)
- component: $CI_SERVER_FQDN/platform-engineering/poc/components/maven-component/gitlab-ci-maven@0.3.0
  inputs:
    image: gitlab-registry.insee.fr:443/kubernetes/images/build/maven-jdk:21
    deploy-enabled: true
```

FIGURE 2 – Extrait de fichier ".gitlab-ci.yml" avec et sans Component
 À gauche, la capture d'écran montre un fichier ".gitlab-ci.yml" sans component, avec le job Maven écrit en dur dans le fichier.
 À droite, la capture d'écran montre le même fichier ".gitlab-ci.yml" avec component, qui remplit donc la même fonction mais en étant bien moins verbeux.

5.3 Solution retenue et architecture de l'usine logicielle

L'adoption du paradigme To Be Continuous au sein de notre GitLab interne est une option qui semble intéresser l'équipe KubeApp.

Plutôt que d'épuiser nos ressources internes à maintenir des scripts de build ou des composants maison, ce choix stratégique permet de déléguer la maintenance du "moteur" de CI/CD à une communauté experte, tout en concentrant les efforts de nos équipes sur l'assemblage et la gouvernance.

- **Inclusion synthétique** : Dans le fichier .gitlab-ci.yml du projet, le développeur n'écrit plus des lignes de scripts complexes. Il inclut simplement les briques (composants) dont son projet a besoin (ex : Java/Maven, Docker, SonarQube, ...).
- **Exécution complète et standardisée** : Chaque component respecte les bonnes pratiques du génie logiciel (sécurité à chaque étape, traçabilité, tests, ...) par défaut. Aussi, au sein du projet To Be Continuous, les différents composants sont cohérents entre eux et fonctionnent donc naturellement avec les autres composants (exemple : Kubernetes déploie l'image construite par Docker, qui lui-même récupère les artefacts produits par Maven. SonarQube récupère les rapports de couverture de Maven, ...).
- **Maintenance déléguée** : En plus d'avoir des composants complets, la maintenance est assurée par une communauté très présente et réactive. Ainsi, l'équipe plateforme sera libérée de ce poids et sera donc plus disponible pour ses activités essentielles sur la plateforme.

Cette délégation ne se fait cependant pas au détriment du contrôle : l'équipe plateforme conserve la main sur le choix des composants intégrés au Golden Path et sur leur version, ce qui lui permet de garantir la stabilité de l'usine logicielle tout en bénéficiant des mises à jour de la communauté à son propre rythme.

METTRE UN COMPARATIF D'UN COMPONENT INSEE VS UN COMPONENT TBC, MAIS COMMENT FAIRE ? (UN SCREEN N'EST PAS ADAPTÉ. UN TABLEAU COMPARATIF PEUT-ÊTRE ?)

5.4 Implémentation du pipeline standardisé

Pour démontrer la valeur de cette approche dans le cadre du Golden Path, nos travaux de mise en œuvre se sont concentrés sur la validation et l'intégration du composant d'une chaîne Java/Maven (le composant Maven Build de To Be Continuous), tout en posant la structure du pipeline standard cible.

De la même manière que le template d'initialisation de projet incarnait le Golden Path lors de la création, ce pipeline standardisé en constitue le prolongement naturel : il garantit que les bonnes pratiques intégrées à la création du projet restent appliquées à chaque évolution ultérieure du code.

L'implémentation de cette usine logicielle standardisée s'articule autour de trois piliers :

- **Le Build et la Qualité de Code automatisée** : Dès qu'un développeur pousse son code, le pipeline déclenche la compilation et les tests unitaires. Le composant communique directement avec notre instance SonarQube pour analyser la qualité du code (couverture de tests, dette technique, métriques de complexité, ...) et applique une Quality Gate. Si le code ne respecte pas les critères de qualité, le pipeline s'interrompt.
- **La Sécurité intégrée au plus tôt (Shift Left Security)** : Au lieu d'attendre un audit de sécurité tardif, le pipeline embarque automatiquement des scanners de vulnérabilités. Il contrôle la présence de failles connues dans les dépendances applicatives (SCA : Software Composition Analysis) et dans le code source (SAST : Static Application Security Testing).
- **Le Packaging et la traçabilité** : Une fois le code validé et testé, le pipeline procède à la construction de l'image de conteneur, crée un tag de l'image et la publie dans le registre d'images sécurisé.

Ces trois piliers partagent un même principe organisationnel : déplacer le contrôle qualité et sécurité le plus tôt possible dans le cycle de vie du projet, là où le coût de correction d'une anomalie est le plus faible, plutôt que de le découvrir tardivement en production.

5.5 Évaluation théorique : Fiabilité, conformité et valeur pour la DSI

Réponse aux besoins des développeurs (DevEx)

Pour les équipes de développement, la CI/CD cesse d'être une corvée technique obscure. Le fichier de configuration de leur pipeline se résume à quelques lignes déclaratives. Ils bénéficient d'un feedback rapide et clair directement sur GitLab en cas d'erreur ou de faille détectée. De plus, ils n'ont plus à se préoccuper de la mise à jour des scripts de build : la montée en version des outils est gérée de manière transparente au niveau des composants.

Ce gain de temps n'est pas seulement au niveau individuel : à l'échelle de la DSI, il représente une capacité de développement significative qui, plutôt que de se concentrer sur des tâches d'infrastructure répétitives, s'oriente vers la production de valeur métier.

Réponse aux exigences d'ingénierie et de gouvernance (DSI & Sécurité)

Pour la DSI, l'adoption de composants centralisés de TBC offre une garantie de sécurité et de conformité par défaut.

L'équipe SSI a l'assurance que chaque projet utilisant le Golden Path applique rigoureusement les contrôles de sécurité requis, sans dépendre de la disponibilité ou du niveau d'expertise de chaque équipe. Enfin, sur le plan opérationnel, l'INSEE réduit drastiquement la charge de maintenance de son usine logicielle en s'appuyant sur l'effort communautaire, permettant aux équipes transverses de se consacrer à l'accompagnement et à l'amélioration de la plateforme.

Cette standardisation transforme également la nature des échanges entre les équipes applicatives et les équipes de sécurité : les échanges portent désormais sur les dérogations exceptionnelles à un standard commun, plutôt que sur la vérification répétée de pratiques de base projet par projet.

6 Étape 3 : Le Déploiement Continu et l'Infrastructure

L'étape de déploiement peut être vu comme l'intersection entre le monde des développeurs et celui de la production. C'est traditionnellement à cet endroit que se concentrent les plus forts blocages organisationnels et la plus grande complexité technique.

C'est également l'étape où le Golden Path prend tout son sens : après avoir sécurisé la naissance du projet et la fabrication de l'artefact, il reste à garantir que sa mise en production ne redevienne pas, à son tour, une source d'hétérogénéité et de friction.

6.1 Expression du besoin : La complexité du déploiement et le défi de l'hétérogénéité des infrastructures

Le déploiement d'une application moderne exige aujourd'hui des compétences approfondies en infrastructure. Pour un développeur applicatif, livrer un service en production ne consiste plus simplement à déposer un fichier sur un serveur. En effet, il faut désormais orchestrer des conteneurs, paramétrer des *Health Checks*, configurer des routes réseau, gérer des certificats SSL, provisionner des bases de données réparties, etc.

La fracture de la charge cognitive Kubernetes et Cloud Native

L'adoption de Kubernetes au sein de notre DSI a apporté un gain significatif en termes d'élasticité et de résilience. Cependant, elle a également imposé une charge cognitive considérable aux équipes applicatives. La syntaxe des manifestes YAML Kubernetes (réputée verbeuse, stricte et complexe) exige une expertise Ops que la majorité des développeurs ne possèdent pas (et ne souhaitent pas acquérir au détriment de leur métier). Cette complexité entraînait alors des erreurs récurrentes de configuration, une opacité sur l'état réel des déploiements et une **dépendance systématique aux administrateurs de clusters**.

Pour pallier à ce problème, l'équipe KubeApp a créé le Chart Générique. Il s'agit d'un Chart Helm pour déployer facilement une application sur Kubernetes. Bien que cette initiative ait réduit la charge mentale des développeurs, ces derniers doivent malgré tout manipuler des fichiers YAML pour utiliser le Chart Générique. Ainsi, bien que la syntaxe soit grandement simplifiée par rapport aux manifestes YAML bruts, les équipes de développement doivent toujours faire face à un format complexe pour des profils applicatifs purs lorsqu'ils ont besoin d'adapter le Chart Générique ou de faire des modifications de configuration.

Le Chart Générique a ainsi constitué une première avancée, mais elle a surtout mis en lumière la nature du problème restant : ce n'est plus tant la complexité de Kubernetes en tant que telle qui freine les équipes, que le simple fait de devoir écrire et maintenir du YAML pour exprimer un besoin métier.

La cohabitation des trois mondes : Machines Virtuelles, Kubernetes et Cloud

De plus, notre SI ne se résume pas à un unique environnement uniforme. En effet, trois paradigmes d'infrastructure cohabitent :

- **Le monde des Machines Virtuelles (VM)** : Très présent au sein de notre DSI, il héberge le patrimoine applicatif historique (legacy). Bien que la stratégie globale vise une modernisation progressive vers le conteneur, une grande partie de ces applications ne peut être migrée à court terme pour des raisons de disponibilité critique, de coût d'héritage ou d'effort de refactorisation.
- **Le monde Kubernetes** : Représente le standard cible pour l'ensemble des nouveaux développements et des applications réarchitecturées.
- **Le monde Cloud (Public/Hybride)** : A pour objectif d'offrir des services managés (bases de données PaaS, stockages objets) consommés à la demande.

L'enjeu majeur du Golden Path de déploiement consistait à privilégier l'accélération sur le monde Kubernetes et Cloud, là où la valeur ajoutée et le potentiel d'agilité sont les plus élevés, sans chercher à bloquer l'initiative en voulant résoudre d'emblée l'ensemble des contraintes du monde VM. La ligne directrice retenue était claire : si nos avancées de plateforme bénéficient indirectement au monde VM, tant mieux, mais la priorité absolue reste la simplification de l'écosystème Cloud Native.

Étape du déploiement	Décisions & Choix imposés	Frictions, Tickets & Charge cognitive
1. Choix de la cible d'hébergement	VM historique (Legacy), cluster Kubernetes ou offre Cloud	Avis de cadrage auprès des architectes/Ops, recherche de documentation éparpillée et non unifiée.
2. Définition de la configuration	Rédaction des fichiers YAML (manifestes Kube ou <i>values.yaml</i> du Chart Générique).	Surcharge cognitive liée au vocabulaire Kube (Ingress, Probes, Resources Limits), erreurs de syntaxe et « copier-coller » depuis d'anciens projets.
3. Provisionnement Réseau & Sécurité	Ouverture des flux firewall, enregistrement DNS, gestion des certificats SSL/TLS.	Ouverture de multiples tickets auprès des équipes transverses, délais d'attente importants et blocages au moment du déploiement.
4. Provisionnement de l'Infrastructure	Création des bases de données, comptes de service et espaces de stockage	Dépendance directe envers les experts Ops pour la création des instances, retards d'attribution et gestion manuelle des identifiants/secrets.
5. Déploiement & Diagnostic	Exécution du déploiement et vérification du bon fonctionnement en recette/prod.	Manque de visibilité sur l'état réel des Pods/VMs, investigation complexe dans des outils de logs séparés, dépendance aux Ops en cas de plantage.

TABLE 3 – Cartographie du parcours développeur lors du déploiement traditionnel (Avant Golden Path)

6.2 Panorama des technologies et étude des approches d'abstraction

Pour masquer la complexité d'infrastructure et offrir une expérience de déploiement fluide, plusieurs architectures et outils ont été étudiés :

Ces différentes pistes ont été évaluées selon trois axes : le niveau d'abstraction réellement offert au développeur, la capacité de la DSI à faire évoluer les règles de déploiement de façon centralisée dans le temps, et la maturité de la solution pour un usage en production.

- **Le Chart Helm Générique (Initiative interne existante)** : L'équipe KubeApp avait conçu un Chart Helm mutualisé. Bien qu'il ait permis une première standardisation en évitant à chaque équipe d'écrire ses propres définitions Kubernetes, son utilisation nécessitait toujours la manipulation manuelle de fichiers de valeurs *values.yaml* complexes.
- **Le Scaffolding Backstage pur** : Cette solution consistait à faire générer par Backstage les fichiers de configuration lors de la création du projet. Cependant, cette approche présentait un risque de "dérive de configuration" (drift) : une fois le fichier généré, il devenait difficile pour la DSI de mettre à jour globalement les règles de déploiement sur l'ensemble des dépôts.
- **Kratix** : Solution open-source innovante basée sur le concept de "Promises" pour construire des plateformes. Bien que séduisante par sa capacité à orchestrer indifféremment des VM et des conteneurs, sa maturité pourrait être incompatible avec notre démarche.
- **Crossplane** : Framework de contrôle permettant d'étendre l'API Kubernetes pour provisionner et gérer n'importe quelle ressource (Cloud, bases de données, clusters) à l'aide de définitions de ressources personnalisées (XRD). Crossplane permet aux équipes Ops d'exposer des briques d'infrastructure sous forme d'APIs simplifiées et sécurisées (en particulier, dans notre écosystème, Crossplane va pouvoir se connecter aux API déjà existantes de la DSI).

Ce panorama fait apparaître une distinction essentielle entre deux familles de solutions : celles qui **génèrent** de la configuration à un instant T (Chart Générique seul, Scaffolding pur), et celles qui **maintiennent** cette configuration dans le temps via un mécanisme de réconciliation continue (Kratix, Crossplane). C'est cette seconde famille qui a concentré l'essentiel des travaux de comparaison, qui seront détaillés dans la sous-section suivante.

6.3 Arbitrage : Backstage vs Kratix vs Crossplane

Une partie importante de mon stage fut dédiée à étudier et à comparer Backstage, Kratix et Crossplane par rapport à ce qu'ils pourraient apporter à notre Golden Path. Cette étude, menée sur deux cas d'usage complémentaires (la modification d'une application déjà déployée sur Kubernetes, puis la modification d'une ressource Cloud ou d'une VM existante), est disponible en intégralité en Annexe 9. Cette sous-section n'en présente que la synthèse décisionnelle.

Backstage seul a été écarté en premier. Cette approche offre l'expérience développeur la plus simple à mettre en place, mais ne repose sur aucune garantie technique : les règles de configuration n'existent que dans le formulaire et peuvent donc être contournées (modification manuelle par exemple). De plus, la charge de travail de l'équipe plateforme augmente avec chaque nouvelle équipe **servie**, ce qui en fait une solution difficile à tenir dans la durée à l'échelle de la DSI.

La combinaison Crossplane + Kratix a également été écartée car elle cumule la charge d'apprentissage et de maintenance des deux outils, pour un bénéfice qui excède les besoins de notre premier périmètre. Cette option reste une évolution possible si nos besoins de gouvernance venaient à se complexifier.

L'arbitrage s'est donc concentré sur Kratix et Crossplane. Les deux solutions apportent une réponse robuste à l'hétérogénéité de nos infrastructures, mais selon des logiques différentes. Kratix est capable d'orchestrer indifféremment des outils déjà en place, quel que soit le monde visé (VM, Kubernetes, Cloud), ce qui en fait un candidat particulièrement pertinent pour notre parc où les VM sont encore très présentes. Crossplane, à l'inverse, apporte une réconciliation continue et immédiate sur les ressources qu'il gère, avec un support mature et directement opérationnel sur les écosystèmes Kubernetes et Cloud.

Or, comme établi dans l'expression du besoin de cette section, le Golden Path de déploiement a concentré son premier périmètre sur les mondes Kubernetes et Cloud, en traitant le monde VM comme un enjeu secondaire à ce stade. Sur ce périmètre restreint, l'atout de Kratix (l'orchestration transparente d'outils hétérogènes, notamment sur VM) perd une grande partie de sa pertinence, tandis que Crossplane y dispose justement de ses intégrations les plus matures. Par ailleurs, la validation humaine nécessaire avant une mise en production reste assurée par le mécanisme de revue de code (Merge Request) déjà en place dans notre chaîne, ce qui rend la richesse des workflows Kratix moins déterminante à ce stade.

C'est cette combinaison de facteurs qui a fait pencher la balance en faveur de **Crossplane** comme brique d'abstraction pour la première itération de notre Golden Path de déploiement. Le tableau ci-dessous synthétise cet arbitrage à l'échelle de l'INSEE. Le détail des critères techniques et des scénarios étudiés figure en Annexe 9.

Critère	Backstage seul	Crossplane (re-tenu)	Kratix	Crossplane + Kratix
Expérience développeur	Très simple (formulaire), mais aucun garde-fou technique derrière.	Identique (formulaire), avec un contrôle technique systématique en plus.	Identique (formulaire), avec une étape de validation explicite avant la mise en production.	Identique aux deux solutions précédentes.
Niveau de gouvernance apporté	Faible : les règles n'existent que dans le formulaire et restent contournables.	Fort : toute configuration non conforme est rejetée, et toute dérive est corrigée automatiquement, en continu.	Très flexible sur les processus (approbations, vérifications métier), mais la correction d'une dérive n'est pas immédiate.	Le plus complet : contrôle technique et processus métier combinés.
Couverture des mondes VM / Kubernetes / Cloud	Fonctionne sur les trois mondes, mais sans mutualisation : chaque monde nécessite son propre outillage dédié.	Très robuste sur Kubernetes et Cloud (support mature), mais support du monde VM encore jeune.	Capable d'orchestrer les outils déjà en place sur les trois mondes sans attendre la maturité d'un support dédié.	Cumule les forces des deux solutions, au prix d'une coordination plus lourde entre elles.
Effort d'adoption pour l'équipe plateforme	Faible au démarrage, mais croît avec le nombre d'équipes et de types d'applications à couvrir.	Investissement initial significatif (nouvel outil, nouvelles compétences), stable ensuite quel que soit le nombre d'équipes.	Investissement comparable à Crossplane, avec en plus le développement et la maintenance de la logique métier.	Le plus élevé : cumul des deux investissements, plus l'effort de cohérence entre les deux outils.
Capacité à passer à l'échelle	Limitée : la charge augmente avec chaque nouvelle équipe servie.	Bonne : l'effort de conception est mutualisé pour l'ensemble des équipes.	Bonne, pour les mêmes raisons.	Bonne, mais avec une complexité de maintenance plus élevée.
Verdict pour notre premier périmètre	Écarté : Insuffisant pour garantir la conformité à l'échelle de la DSI.	Solution retenue : Meilleur compromis entre robustesse technique et maturité, sur le périmètre priorisé.	Écarté à ce stade : Sa capacité à unifier des outils sur les trois mondes pèse moins sur notre périmètre.	Écarté : Complexité jugée trop importante par rapport aux besoins du premier périmètre.

TABLE 4 – Synthèse organisationnelle de l'étude d'opportunité Backstage / Crossplane / Kratix

6.4 Architecture de déploiement cible : Le modèle GitOps avec GitLab CI et ArgoCD

Pour l'orchestration des déploiements, nous avons retenu le paradigme GitOps, devenu l'état de l'art pour la gestion d'infrastructure et d'applications conteneurisées.

Le principe fondamental du GitOps est le suivant : Git est la source unique de vérité (Single Source of Truth). L'état désiré du système est décrit de manière déclarative dans un dépôt Git dédié, et un agent autonome se charge de synchroniser en continu l'état réel du cluster avec cet état désiré.

Découpage strict des responsabilités : GitLab CI vs ArgoCD

Dans notre architecture cible, un découpage étanche est opéré entre la chaîne de CI et la chaîne de CD :

- **GitLab CI (L'Usine de Build)** : S'occupe exclusivement de la compilation, des tests de qualité/sécurité, de la création de l'image de conteneur et du push vers le registre. Une fois l'image créée, le pipeline GitLab CI met automatiquement à jour le numéro de version (tag) dans le dépôt GitOps. Il n'interagit jamais directement avec le cluster Kubernetes de production.
- **ArgoCD (L'Orchestrateur de Déploiement)** : Installé directement au cœur du cluster Kubernetes, ArgoCD surveille en permanence le dépôt GitOps. Dès qu'une modification est détectée, il applique les configurations sur le cluster.

Ce modèle apporte des bénéfices majeurs : suppression des accès directs des pipelines aux clusters (sécurité renforcée), traçabilité absolue des changements via les commits Git, et capacité de revenir à une version antérieure (rollback) en un simple clic Git.

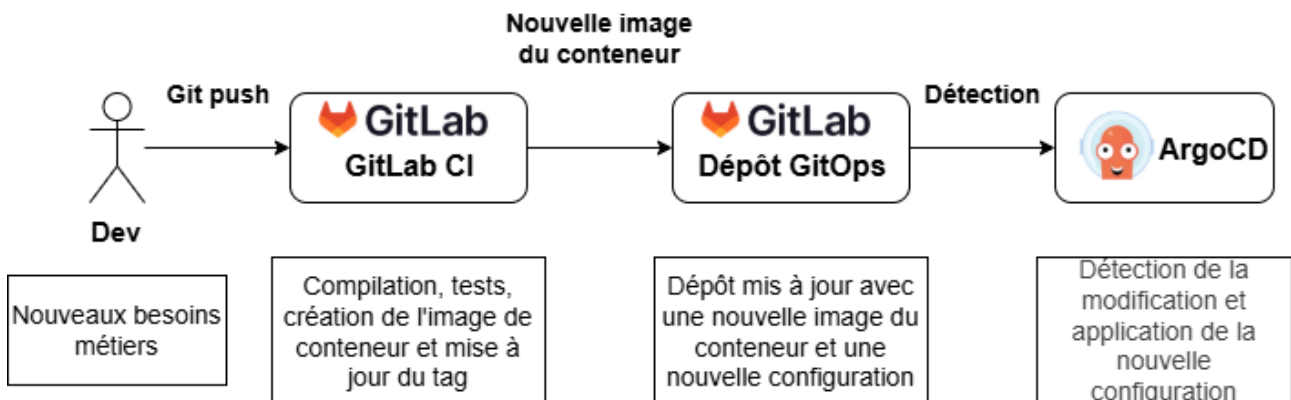


FIGURE 3 – Vue macroscopique du modèle GitOps

6.5 Réalisation du PoC : L'alliance Backstage, Crossplane et ArgoCD

Afin de valider la faisabilité du Golden Path de déploiement sans imposer de complexité aux développeurs, nous avons conçu et mis en œuvre un PoC.

Ce PoC prouve qu'il est effectivement possible d'offrir une expérience de déploiement entièrement abstraite, où le développeur n'a plus jamais à rédiger le moindre manifeste Kubernetes ni le moindre fichier Helm.

Le déroulement du workflow utilisateur :

1. **Interface Simplifiée (Backstage)** : Sur la page de son application dans le portail Backstage, le développeur clique sur un bouton "Modifier la configuration". Un formulaire simple lui permet de paramétrer ses besoins métier : taille de l'application (ex : Small, Medium, Large), nombre de répliques souhaité, et variables d'environnement.
2. **Abstraction de la demande (Crossplane)** : La validation du formulaire dans Backstage ne génère pas du YAML Kubernetes brut, mais instancie une ressource Crossplane (Composite Resource Claim). Crossplane traduit cette demande simple en s'appuyant sur notre Chart Générique Helm configuré par l'équipe KubeApp.
3. **Déploiement Automatisé (ArgoCD)** : La configuration générée est commitée automatiquement dans le dépôt GitOps du projet. ArgoCD détecte la mise à jour et applique la nouvelle topologie sur le cluster Kubernetes en quelques secondes.

Grâce à ce workflow, la complexité de Kubernetes est totalement masquée. Le développeur manipule une interface métier claire, tandis que l'ingénierie Ops conserve la maîtrise des configurations sous-jacentes via Crossplane et le Chart Générique.

Ce PoC ne couvre volontairement qu'un sous-ensemble limité de champs métier (taille, répliques, variables d'environnement). L'objectif n'était pas de couvrir tous les cas d'usage dès cette étape, mais de démontrer que le principe d'abstraction fonctionne de bout en bout. L'élargissement du formulaire à d'autres besoins (exposition réseau, bases de données, ...) constitue la suite logique des travaux, une fois cette première brique validée.

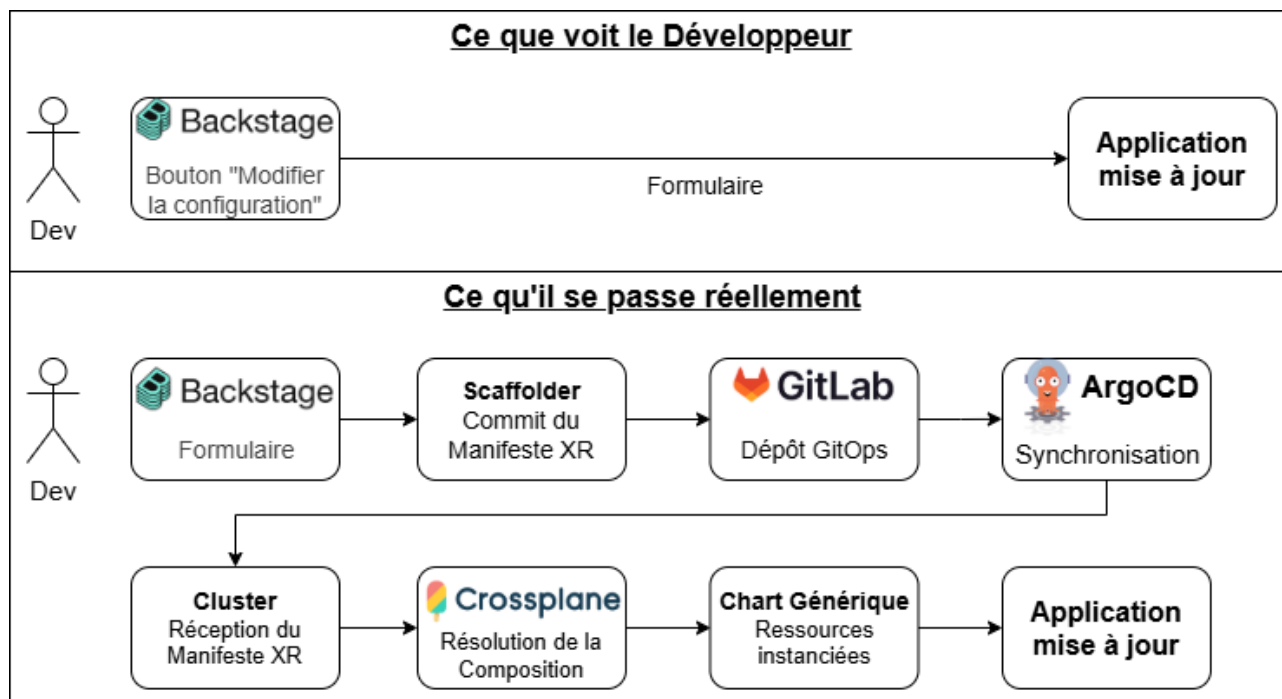


FIGURE 4 – Vue macroscopique de la chaîne : Point de vue développeur vs Réalité

6.6 Bilan et valeur stratégique pour la DSI

Le PoC mis en place prouve qu'il est possible de concilier la liberté du développeur avec le niveau de contrôle exigé par une DSI moderne.

Réponse aux besoins des développeurs (DevEx)

Pour les équipes de développement, le processus de déploiement passe d'une tâche anxiogène et complexe à une action en libre-service, intuitive et instantanée. L'intégration de la vision ArgoCD dans Backstage leur permet de suivre le statut de leur application en temps réel sans avoir à utiliser d'outils de ligne de commande Kubernetes (*kubectl*).

Cette bascule vers le libre-service a également un effet organisationnel indirect : elle réduit le nombre de tickets ouverts auprès des équipes transverses pour de simples ajustements de configuration, libérant du temps aussi bien côté développeurs (moins d'attente) que côté plateforme (moins de sollicitations répétitives).

Réponse aux exigences d'infrastructure et de sécurité (DSI)

Pour la DSI, cette architecture GitOps pilotée par Crossplane permet de poser les bases d'un socle d'infrastructure évolutif. Les équipes Ops ne sont plus sollicitées pour exécuter des déploiements ou corriger des erreurs de syntaxe YAML. Elles concentrent leur travail sur la maintenance du Chart Générique et des compositions Crossplane, garantissant ainsi que l'ensemble des applications déployées en production respectent scrupuleusement les exigences de résilience, de sécurité et d'observabilité internes.

Cette approche prolonge directement le principe de *Compliance by Design* déjà évoqué pour l'initialisation de projet (section 4) : la conformité n'est plus vérifiée après coup, mais garantie structurellement par la Composition elle-même, à chaque déploiement.

Enfin, ce socle n'est pas figé : comme détaillé dans l'étude d'opportunité en annexe, le périmètre retenu ici couvre volontairement Kubernetes et le Cloud en priorité. L'extension éventuelle de la démarche au monde des VMs, ou l'enrichissement de la gouvernance via des workflows plus élaborés, restent des trajectoires documentées et disponibles si les besoins de la DSI évoluent en ce sens.

7 Étape 4 : Le Point d'Accès Unique : L'Internal Developer Platform

Après avoir adressé les enjeux de création de code, d'industrialisation de l'usine logicielle et d'automatisation des déploiements, se pose la question de l'accessibilité et de la gouvernance globale. Sans une interface unifiée pour orchestrer ces briques, le système risque de demeurer un assemblage complexe d'outils hétérogènes. C'est tout le rôle de l'Internal Developer Platform (IDP) et de son portail.

Cette dernière étape agit donc surtout comme un **liant** organisationnel plutôt qu'une brique technique à part entière. Elle unifie et rend cohérent tout ce qui a été construit dans les étapes précédentes du Golden Path.

7.1 Expression du besoin : Combattre la dispersion et créer la "Single Source of Truth"

Avant la mise en place d'un portail développeur, l'écosystème d'outils au sein de notre DSI était très dispersée. Pour mener à bien ses activités quotidiennes ou assurer le maintien en condition opérationnelle (MCO) de son application, un développeur devait naviguer entre plusieurs d'interfaces déconnectées telles que :

- Les différentes documentations propres à chaque équipe pour chercher une documentation souvent obsolète.
- GitLab pour accéder au code source et suivre les pipelines de CI/CD.
- La gestion de tickets pour suivre les demandes d'accès ou les incidents.
- Diverses consoles d'infrastructures (ArgoCD, Grafana, Rundeck, ...) pour surveiller le comportement des applications en production.

Cette dispersion entraînait un coût d'efficacité important. Les développeurs consacraient un temps précieux à chercher qui possédait tel composant, où se trouvait l'API d'un service voisin ou quelle était la cause d'un dysfonctionnement en recette.

Ces frictions représentent une perte de productivité collective difficile à chiffrer précisément, mais dont l'impact est largement prouvé et ressenti : perte de contexte entre les outils, découragement face à la complexité perçue, et manque de vision globale.

Il est donc très important d'établir une Source Unique de Vérité (Single Source of Truth) : un portail centralisé offrant une visibilité complète sur le patrimoine applicatif de la DSI et simplifiant les interactions avec le reste du SI.

7.2 Étude comparative des solutions de portail développeur

Pour bâtir un point d'accès unifié capable de répondre aux irritants observés, trois grands paradigmes d'ingénierie ont été évalués par la DSI.

Cette analyse a été menée en confrontant chaque solution à un même besoin : offrir une expérience unifiée et pérenne, sans faire peser sur l'équipe plateforme une charge de maintenance disproportionnée par rapport au bénéfice apporté.

- **Les portails documentaires légers** : Différentes équipes à l'INSEE ont créé des applications spécifiquement pour pouvoir retrouver rapidement la documentation dont les développeurs ont besoin. Ces initiatives, bien que très inspirantes, sont néanmoins souvent légères (par exemple : une page web avec un lien vers les pages documentaires existantes) et, même si le portail est à jour, dépendent de la qualité et de la tenue à jour des différentes documentations référencées.
- **Le développement d'un portail interne "Maison"** : Coder une interface sur-mesure offre une liberté totale au démarrage, mais représente un piège de maintenance majeur. L'effort de MCO (développement frontend, backend, connecteurs) consommerait une part importante des ressources de l'équipe Plateforme au détriment de l'ingénierie d'infrastructure.
- **Un cadre open-source standardisé : l'Internal Developer Platform (IDP)** : S'appuyer sur une plateforme open-source éprouvée par l'industrie permet de bénéficier d'une architecture extensible et de concentrer l'effort sur l'intégration de nos propres briques techniques.

Cette comparaison montre que, à l'échelle de l'INSEE, ni la juxtaposition d'initiatives locales ni le développement d'un outil sur-mesure ne pouvaient répondre durablement au besoin exprimé. C'est ce constat qui a orienté nos travaux vers l'étude des solutions IDP existantes sur le marché, dont l'arbitrage est détaillé plus loin dans cette section.

7.3 Le concept d'Internal Developer Platform (IDP)

Avant de retenir une technologie spécifique, il convient de définir ce qu'est une **Internal Developer Platform (IDP)**. Une IDP ne se résume pas à un simple agrégateur de liens hypertexte ou à un tableau de bord : il s'agit d'une couche d'abstraction au-dessus de l'ensemble de l'outillage de la DSI.

Elle a pour rôle d'orchestrer les systèmes sous-jacents (infrastructure, CI/CD, sécurité, gestion des identités) afin d'offrir aux développeurs un accès en libre-service (**Self-Service**) à l'ensemble des ressources du SI, tout en appliquant les règles de gouvernance de la DSI de manière transparente. Le portail développeur en constitue la vitrine applicative et le point d'interaction quotidien.

Cette définition rejoint directement la philosophie du Golden Path développée tout au long de ce rapport. L'IDP n'est pas une fin en soi, mais le point de convergence qui rend visible et accessible l'ensemble des briques (Scaffolder, usine logicielle, déploiement) déjà présentées dans les étapes précédentes.

7.4 Arbitrage : le choix de l'outil IDP

Une fois le paradigme d'un cadre IDP open-source standardisé retenu, il restait à choisir l'outil concret pour l'implémenter. Cet arbitrage, mené en confrontant les principaux acteurs du marché à nos contraintes propres, est détaillé intégralement en Annexe 12. Cette sous-section n'en présente que la synthèse décisionnelle.

L'évaluation s'est appuyée sur des critères directement liés à notre contexte INSEE : la possibilité d'un hébergement on-premise pour des raisons de souveraineté sur nos données, l'absence de dépendance forte à un éditeur (*vendor lock-in*), un coût maîtrisé qui ne croît pas avec le nombre de développeurs (+ de 600 au sein de la DSI), la compatibilité avec notre outillage déjà en place (GitLab interne, ArgoCD, Helm, Vault, Kubernetes), et la capacité à adapter l'outil à nos spécificités internes.

Deux familles de solutions ont été écartées d'emblée. D'une part, les offres liées à un hyperscaler (Microsoft DevBox, AWS Proton) imposent un environnement Azure ou AWS, ce qui est incompatible avec notre contexte on-premise. D'autre part, des solutions comme Pulumi, davantage centrées sur l'Infrastructure as Code que sur l'expérience développeur globale, ne couvrent pas les besoins fonctionnels d'un IDP complet (pas de portail, pas de catalogue de services, pas de documentation centralisée).

L'arbitrage s'est donc concentré sur quatre candidats sérieux :

- **GitLab** : Déjà déployé au sein de la DSI, présentait l'avantage de ne pas ajouter un nouvel outil au paysage existant. Mais sa couverture fonctionnelle reste centrée sur le CI/CD : ni catalogue de services complet, ni documentation technique unifiée, ni équivalent du Scaffolder ne sont disponibles nativement.
- **Port** : Solution commerciale à l'approche "no-code", offrait une mise en place rapide et une interface moderne. Son modèle SaaS uniquement est en revanche incompatible avec nos exigences de souveraineté sur les données sensibles, et son modèle de licences par utilisateur en ferait une solution coûteuse à l'échelle de nos effectifs.
- **Harness** : Plateforme commerciale combinant CI/CD et IDP, elle souffrait des mêmes limites : coût de licence élevé, hébergement SaaS majoritaire, et surtout un recouvrement fonctionnel important avec notre chaîne GitLab CI déjà en place.
- **Backstage** : Solution open-source initialement conçue par Spotify et aujourd'hui hébergée par la CNCF (au niveau de maturité *Incubating*), Backstage s'est distinguée en cochant l'ensemble des critères retenus : hébergement self-hosted, aucun coût de licence récurrent quel que soit le nombre d'utilisateurs, couverture fonctionnelle complète (catalogue, documentation, scaffolding), et un écosystème de plugins permettant une intégration native avec notre outillage existant.

Cette combinaison de facteurs a fait pencher la balance en faveur de Backstage, malgré un investissement d'adoption plus élevé qu'une solution SaaS clé en main. En effet, la construction et la maintenance d'un portail Backstage demandent des compétences de développement (front comme back) au sein de l'équipe plateforme, une charge que nous avons choisi d'assumer pour préserver notre indépendance vis-à-vis d'un éditeur tiers. Le tableau ci-dessous synthétise cet arbitrage à l'échelle de l'INSEE.

Critère	Backstage (re-tenu)	GitLab	Port	Harness
Souveraineté & hébergement	Open-source, self-hosted : contrôle total sur nos données.	Déjà self-hosted chez nous, mais reste un outil tiers avec ses propres contraintes d'évolution.	SaaS uniquement : incompatible avec nos exigences sur les données sensibles.	Principalement SaaS, Option self-hosted limitée.
Coût à l'échelle (600+ développeurs)	Aucun coût de licence récurrent, quel que soit le nombre d'utilisateurs.	Pas de coût supplémentaire (déjà en place), mais ne couvre pas seul le besoin.	Licences par utilisateur : coût élevé à notre échelle.	Modèle commercial coûteux à notre échelle.
Couverture fonctionnelle de l'IDP	Complète : catalogue, documentation centralisée, scaffolding, plugins.	Partielle : Bien sur le CI/CD, mais sans catalogue ni documentation unifiée.	Complète, mais au prix d'un modèle propriétaire.	Complète, avec un recouvrement important avec notre CI/CD déjà en place.
Dépendance à un éditeur	Aucune : Projet open-source sous gouvernance CNCF.	Faible, mais limitée par la roadmap produit de l'éditeur pour les usages IDP.	Forte : Modèle de données propriétaire.	Forte : Suite commerciale fermée.
Effort d'adoption pour l'équipe plateforme	Élevé au démarrage (compétences front/back nécessaires), stable ensuite.	Faible : Outil déjà maîtrisé en interne.	Faible : Mise en place rapide, sans développement.	Modéré, avec un accompagnement commercial disponible.
Verdict	Solution retenue : Seul candidat cochant l'ensemble de nos contraintes de souveraineté, de coût et de couverture fonctionnelle.	Écarté seul : Insuffisant pour couvrir l'ensemble du besoin IDP, mais reste précieux comme brique d'intégration sous-jacente.	Écarté : Incompatible avec nos exigences de souveraineté et de maîtrise des coûts.	Écarté : Coût et redondance avec l'existant.

TABLE 5 – Synthèse organisationnelle de l'arbitrage des solutions IDP

7.5 Choix et justification de Backstage comme IDP

À l'issue de cet arbitrage, le choix de **Backstage** s'est imposé comme une évidence stratégique. Initialement conçu par Spotify et aujourd'hui hébergé par la *Cloud Native Computing Foundation* (CNCF), Backstage s'est imposé comme la référence industrielle des portails développeurs (**Developer Portals**). Bâti sur une architecture modulaire à base de plugins, il est conçu spécifiquement pour abstraire la complexité technique et centraliser l'outillage DevOps.

Ce choix repose sur trois piliers fondamentaux :

- **L'approche "Software Catalog"** : Backstage modélise le SI sous forme de graphe dynamique (Composants, APIs, Ressources, Équipes). Chaque élément possède une propriété claire et un suivi d'état en temps réel, offrant la *Single Source of Truth* recherchée.
- **Un écosystème de plugins riche** : Backstage s'intègre nativement avec l'ensemble des technologies retenues dans notre *Golden Path* (GitLab, Kubernetes, ArgoCD, SonarQube) grâce à des connecteurs open-source maintenus par la communauté.
- **Le paradigme "Docs-like-Code"** : Avec son moteur *TechDocs*, la documentation technique est rédigée en Markdown directement à côté du code source dans le dépôt Git, puis générée et centralisée automatiquement dans le portail Backstage.

Ces trois piliers convergent vers notre objectif de faire de notre portail non pas un outil de plus dans l'écosystème DSI, mais le point d'entrée unique qui rend tous les autres visibles et exploitables.

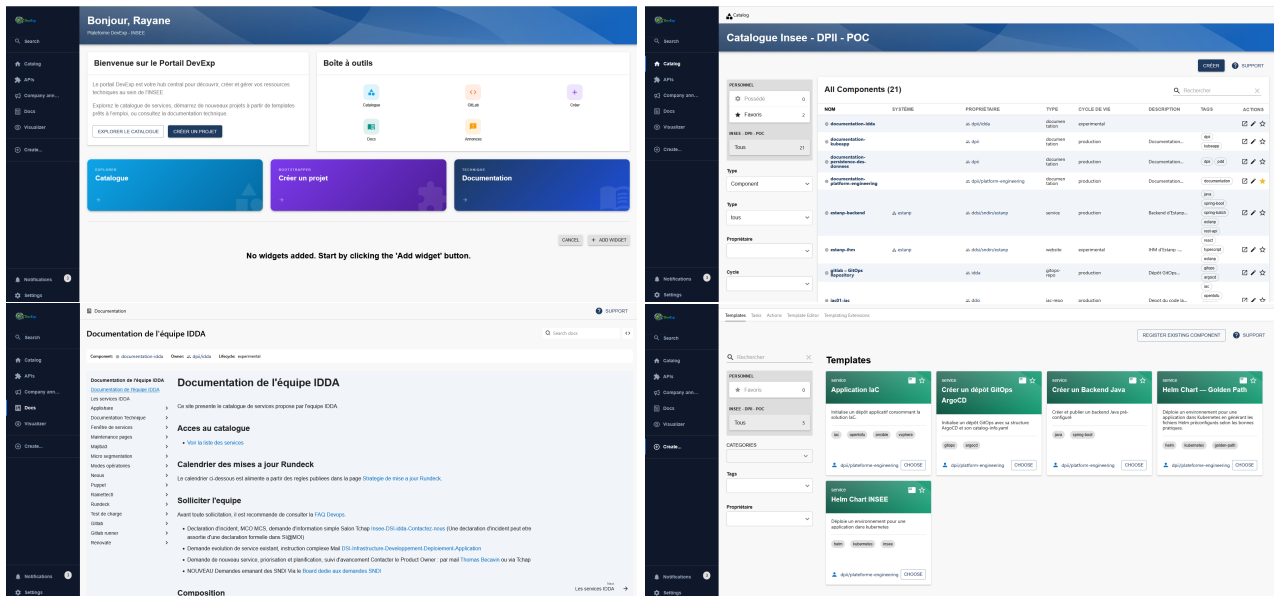


FIGURE 5 – Captures d'écran du portail mis en place avec Backstage

1ère image : Écran d'accueil du portail

2e image : Catalogue des services

3e image : Exemple de documentation accessible

4e image : Templates disponibles

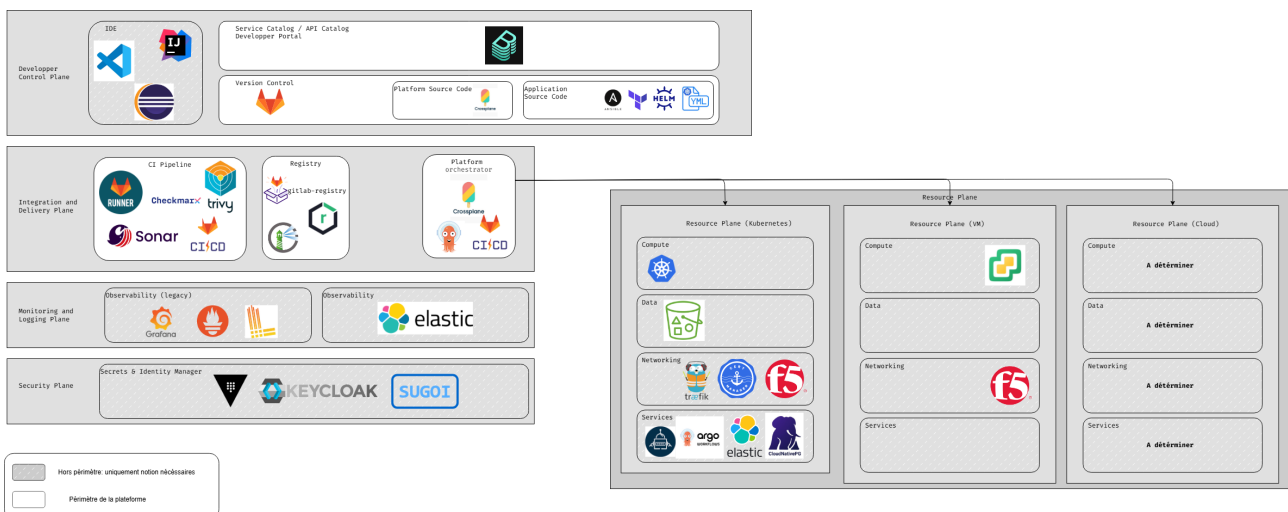


FIGURE 6 – Schéma d'architecture de Backstage au sein du SI de l'INSEE

7.6 Unification du Golden Path au sein du portail

La véritable puissance de notre IDP réside dans sa capacité à unifier l'ensemble des étapes du cycle de vie du logiciel au sein d'un parcours utilisateur fluide et sans couture.

Grâce à cette intégration, le portail Backstage devient le poste de pilotage principal du développeur :

- **Instanciation** : C'est depuis l'interface de Backstage que le développeur déclenche la création d'un nouveau projet via les templates pré-configurés (vus au Chapitre 5). Le service créé est immédiatement enregistré dans le catalogue du portail.
- **Suivi de la Qualité et CI/CD** : Directement sur la fiche de son application dans Backstage, l'ingénieur consulte l'état des pipelines GitLab CI, le résultat des builds, la couverture de code SonarQube et les éventuelles alertes de sécurité sans avoir à ouvrir GitLab.
- **Pilotage du Déploiement** : Grâce aux connecteurs ArgoCD et Kubernetes, le portail pourra afficher en temps réel la santé de l'application sur les différents environnements (Recette, Staging, Production), le nombre de pods actifs et la version actuellement déployée. (Ces connecteurs n'ont pas encore été mis en place)
- **Accès à la connaissance (TechDocs)** : La documentation technique est rendue immédiatement consultable, recherchable et uniformisée pour l'ensemble des projets du SI.

Ce parcours illustre concrètement ce que recouvre la notion de Golden Path : non pas une succession d'outils juxtaposés, mais un fil conducteur unique, du premier clic de création jusqu'au pilotage en production.

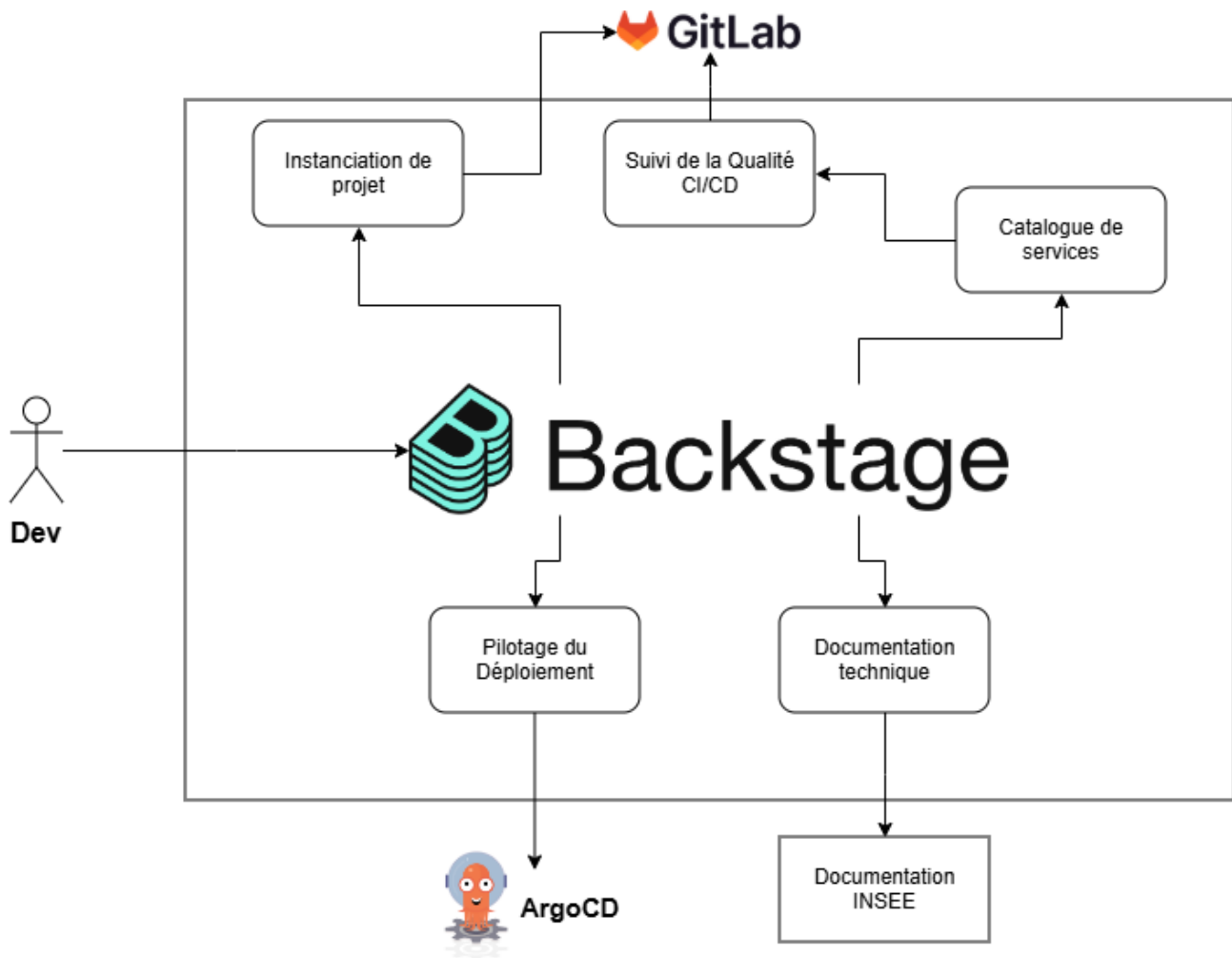


FIGURE 7 – Fonctionnement global de la plateforme du point de vue d'un développeur
Note de lecture :

- Depuis le portail, un développeur peut créer un nouveau projet, surveiller l'état général de ses applications et modifier leurs déploiements.
- Par ailleurs, il a accès, pour tous les services mis à disposition au sein de la DSI, à une documentation complète et recherchable au sein de la plateforme.

7.7 Bilan et valeur ajoutée organisationnelle

L'intégration de ce point d'accès unifié concrétise les promesses du **Platform Engineering** en apportant une réponse globale aux irritants identifiés lors de notre enquête interne.

Comme pour les étapes précédentes du Golden Path, cette valeur ajoutée profite à la fois aux équipes de développement au quotidien et au pilotage stratégique porté par la DSI.

Réponse aux besoins des développeurs (DevEx)

Pour les équipes applicatives, la plateforme met fin au sentiment de dispersion outillée. La charge cognitive est drastiquement réduite : le développeur dispose d'un point d'entrée unique pour créer, suivre et documenter ses composants. La découvrabilité des services existants évite la réinvention d'outils et favorise la réutilisation de briques inter-équipes.

À l'échelle de plusieurs centaines de développeurs, cette réduction de friction quotidienne se traduit par un gain de temps collectif significatif, réinvesti dans la production de valeur plutôt que dans la recherche d'information.

Réponse aux exigences de gouvernance et de pilotage DSI

Pour le management et les équipes transverses, le portail Backstage permet d'avoir une vision d'ensemble du SI. La DSI dispose enfin d'une cartographie exacte et mise à jour de son patrimoine applicatif, identifiant pour chaque service son équipe propriétaire, son niveau de maturité et son état de conformité. Ce niveau de transparence facilite le pilotage stratégique, accélère l'*onboarding* des nouveaux arrivants et garantit une meilleure maîtrise de la sécurité et des coûts à l'échelle de tout le SI.

Ce niveau de visibilité constitue enfin un prérequis à toute démarche d'amélioration continue de la plateforme : on ne peut piloter durablement que ce que l'on peut mesurer et observer.

Conclusion

Bilan du projet et apports personnels

Ce travail de fin d'études visait à répondre à la complexification croissante du SI de l'INSEE et aux irritants quotidiens rencontrés par les équipes applicatives. En plaçant le **Platform Engineering** au cœur de la stratégie de la DSI, l'objectif était de concevoir un cadre unifié capable de réduire la charge cognitive des développeurs tout en garantissant la conformité globale du parc applicatif.

Au terme de ce projet, l'ensemble des objectifs définis en introduction ont été atteints :

- **Cartographie et compréhension fine des besoins** : L'analyse de l'enquête interne a permis d'isoler les points de friction majeurs (opacité des pipelines, dispersion documentaire, dépendance forte aux experts) et de formaliser une matrice de réponse claire.
- **Mise en place de la plateforme IDP et du portail** : L'intégration de Backstage comme point d'entrée unifié a permis de centraliser le catalogue de services, la documentation *Docs-like-Code* et, à terme, le suivi de la santé des applications.
- **Conception du Golden Path** : Le développement du moteur d'initialisation (**Backstage Scaffolder**) offre désormais un mécanisme de création de projets prêts à coder en quelques clics, intégrant nativement les standards de sécurité, de qualité et d'architecture de l'entreprise (*Compliance by Design*).

D'un point de vue personnel et professionnel, ce projet a représenté une excellente opportunité d'apprentissage. Sur le plan technique, il m'a permis de découvrir l'écosystème Kubernetes de l'INSEE et les architectures modernes de portails développeurs. Sur le plan méthodologique, cette expérience m'a permis d'être en lien avec à la fois les mondes Dev et Ops, et donc d'aborder les problématiques spécifiques à chacun. J'ai appris à concevoir un outil interne non pas comme une suite de contraintes techniques, mais comme un produit centré sur ses utilisateurs, nécessitant une écoute constante de l'expérience développeur (**DevEx**) et des compromis permanents entre agilité et gouvernance DSI.

Limites identifiées et conduite du changement

Malgré la valeur ajoutée démontrée par nos prototypes et preuves de concept, le déploiement d'une démarche de **Platform Engineering** se heurte à des défis organisationnels et humains qu'il convient de ne pas sous-estimer.

- **L'accompagnement** : L'adoption d'un nouveau portail ne s'impose pas aux développeurs. Il faudra mener un travail continu auprès des équipes de développement pour leur présenter la plateforme, non comme un outil de contrôle supplémentaire, mais comme un accélérateur de leur quotidien.
- **La conduite du changement et l'inertie des habitudes** : Le principal frein à l'adoption d'une plateforme réside dans les habitudes ancrées, notamment la pratique historique du "copier-coller" de projets ou le contournement des processus standardisés lorsque les équipes sont sous pression de livraison.
- **Le maintien de la flexibilité** : Un risque majeur du **Golden Path** est de transformer ce "chemin doré" en une "cage dorée" (*Golden Cage*). La plateforme devra savoir évoluer pour répondre aux besoins spécifiques d'un projet sans bloquer les initiatives innovantes qui sortiraient temporairement du cadre standard.

Perspectives et évolutions futures

La livraison de ce premier socle pose les bases d'une dynamique à plus longue échelle au sein de la DSI de l'INSEE. Plusieurs axes d'évolution stratégiques se dessinent pour consolider et étendre cette démarche :

- **Enrichissement du Golden Path** : Intégrer de nouvelles dimensions transverses au sein du portail, par exemple le pilotage des ressources déployées, des tableaux de bord d'observabilité avancée et des métriques de sécurité automatisées.
- **Autonomisation de l'infrastructure (*Self-Service Infra*)** : Étendre le périmètre du **Scaffolder** et du portail pour permettre le provisionnement automatisé des ressources d'infrastructure complexes (bases de données managées, files d'attente, réseaux) directement depuis le portail, en s'appuyant sur l'Infrastructure-as-Code.
- **Déploiement global à l'échelle de la DSI** : Généraliser l'usage de la plateforme à l'ensemble des pôles applicatifs de l'organisation, en intégrant le passage par le portail comme l'étape incontournable de tout nouveau projet applicatif.

En conclusion, ce projet démontre que le **Platform Engineering** ne se résume pas à une simple technologie, mais constitue une mutation culturelle et organisationnelle durable. En fournissant aux développeurs un cadre sécurisé et automatisé, l'INSEE se dote d'un levier puissant pour accélérer ses livraisons tout en maîtrisant la complexité de son parc informatique.

9 Annexe 2

11 Annexe 4

12 Annexe 5